

CS336 作业1 (基础)：构建Transformer语言模型

版本 26.0.3

CS336 教学团队

2026年春季学期

1 作业概览

在本次作业中，您将从零开始构建训练标准Transformer语言模型（LM）所需的全部组件，并训练若干模型。

您将实现的内容

1. 字节对编码（BPE）分词器（第2节）
2. Transformer语言模型（LM）（第3节）
3. 交叉熵损失函数与AdamW优化器（第4节）
4. 训练循环，支持模型与优化器状态的序列化与加载（第5节）

您将运行的内容

1. 在TinyStories数据集上训练字节对编码（BPE）分词器。
2. 对数据集执行已训练的分词器编码，将其转换为整数ID序列。
3. 在TinyStories数据集上训练Transformer语言模型。
4. 使用训练好的Transformer语言模型进行样本生成及困惑度评估。
5. 在OpenWebText上训练模型，并将获得的困惑度提交至排行榜。

您可以使用的工具

我们要求您从零开始构建每个组件。特别地，您不得使用以下模块中的任何定义：

`torch.nn`、`torch.nn.functional`或`torch.optim`，但以下例外：

- `torch.nn.Parameter`
- `torch.nn`中的容器类（例如`Module`、`ModuleList`、`Sequential`等）¹
- `torch.optim.Optimizer`基类

您可以使用PyTorch中的其他任意定义。若您想使用某个函数或类，但不确定是否允许，请随时在Slack上提问。如有疑问，请思考使用该功能是否会违背本作业“从零构建”的核心理念。

关于AI工具的声明

AI可完全自主完成作业的许多部分，这使得您更难深入参与并从课程材料中真正获益。

允许使用AI工具解答高层次概念性问题，或提供底层编程文档（例如函数签名、库API）。但禁止使用AI工具实现作业的任何部分，包括编码代理（例如Cursor Agents、Codex、Claude Code）和AI自动补全（例如Cursor Tab、GitHub Copilot）。使用AI代理时，请确保其参考所附AGENTS.md文件，并在使用聊天机器人时一并提交所用提示词。

¹详见pytorch.org/docs/stable/nn.html#containers中的完整列表。

我们强烈建议您在完成作业时，在IDE中禁用AI自动补全功能（例如Cursor Tab、GitHub Copilot）；但非AI类自动补全（例如函数名自动补全）完全允许。往届学生指出，禁用AI自动补全有助于更深入地理解课程内容。

完整AI政策详见该文档。

代码结构说明

作业代码及本说明文档均托管于GitHub，地址为：

github.com/stanford-cs336/assignment1-basics

请使用git clone命令克隆该仓库。若有更新，我们将及时通知您，以便您通过git pull获取最新版本。

1. cs336_basics/*：这是您编写代码的位置。请注意，此处初始为空——您可以完全从零开始自由实现！

2. adapters.py：您的代码必须具备一组特定功能。针对每项功能（例如缩放点积注意力），请通过直接调用您自己的代码来完成其实现（例如run_scaled_dot_product_attention）。注意：您对adapters.py的修改不应包含任何实质性逻辑；该文件仅起胶水代码作用。

3. test_*.py：包含您必须通过的所有测试（例如test_scaled_dot_product_attention），这些测试将调用adapters.py中定义的钩子函数。请勿编辑测试文件。

如何提交

为完成提交，请运行make_submission.sh脚本以生成提交用zip文件。若您有较大的数据文件或检查点文件不希望包含在提交zip中，请务必将其添加至该脚本的排除列表中。

您需向Gradescope提交以下文件：

- writeup.pdf：回答所有书面问题。请使用排版工具撰写您的答案。
- code.zip：包含您编写的所有代码。要提交至排行榜，请向以下地址提交Pull Request：

github.com/stanford-cs336/assignment1-basics-leaderboard

详细提交说明请参见排行榜仓库中的README.md文件。

数据集获取途径

本作业将使用两个预处理数据集：TinyStories [R. Eldan 等, 2023] 和 OpenWebText [A. Gokaslan 等, 2019]。这两个数据集均为单一的大型纯文本文件。

如果您随班级一起完成该作业，可在计算指南中找到下载数据集的相关说明。

如果您在家同步学习，可使用README.md文件中的命令下载这些文件。

低资源提示：初始化

在本课程各项作业说明中，我们将提供如何在GPU资源较少或没有GPU的情况下完成作业部分任务的建议。例如，我们有时会建议缩小数据集或模型规模，或说明如何在Mac上运行训练代码。

集成显卡或CPU。您将在蓝色框（如本框）中找到这些低资源提示。即使您是已注册的斯坦福大学学生，可访问课程专用机器，这些提示仍可能帮助您更快地迭代并节省时间，因此我们建议您阅读！

低资源提示：在Apple Silicon或CPU上运行作业1

借助助教提供的参考实现代码，我们可在配备 Apple M4 Max 芯片及 36 GB 内存的设备上，于 Metal GPU (MPS) 下不到 5 分钟、CPU 下约 30 分钟内训练出一个能生成较流畅文本的语言模型。如果这些术语对你而言尚不熟悉，请不必担心！只需知道：只要你拥有一台较新式的笔记本电脑，且你的实现正确高效，便能够训练出一个小型语言模型，用以生成具有一定流畅度的儿童故事。

作业后续部分将说明若使用 CPU 或 MPS 时需做出哪些调整。

2 字节对编码 (BPE) 分词器

在本作业的第一部分中，我们将训练并实现一种字节级字节对编码 (BPE) 分词器 [R. Sennrich 等人, 2016; C. Wang 等人, 2019]。具体而言，我们将任意 (Unicode) 字符串表示为字节序列，并在此字节序列上训练我们的 BPE 分词器。之后，我们将使用该分词器将文本 (字符串) 编码为词元 (整数序列)，用于语言建模。

2.1 Unicode 标准

Unicode 是一种文本编码标准，将字符映射为整数码点。截至 Unicode 17.0 版本 (2025 年 9 月发布)，该标准共定义了 159,801 个字符，涵盖 172 种文字。例如，字符 s 的码点为 115 (通常记作 U+0073，其中 U+ 为约定前缀，0073 是 115 的十六进制表示)，而字符 牛 的码点为 29275。在 Python 中，可使用 `ord()` 函数将单个 Unicode 字符转换为其整数表示；`chr()` 函数则将整数 Unicode 码点转换为对应的字符串。

```
>>> ord('牛')
29275
>>> chr(29275)
'牛'
```

题目 (unicode1) : 理解 Unicode (1 分)

(a) `chr(0)` 返回哪个 Unicode 字符？提交要求：

用一句话作答。

(b) 该字符的字符串表示形式 (`__repr__()`) 与其打印形式有何不同？

提交要求：用一句话作答。

(c) 当该字符出现在文本中时会发生什么？在 Python 解释器中尝试运行以下代码，观察结果是否符合你的预期：

```
>>> chr(0)
>>> print(chr(0))
```

```
>>> 'this is a test' + chr(0) + 'string'
>>> print('this is a test' + chr(0) + 'string')
```

提交要求：用一句话作答。

2.2 Unicode 编码

尽管 Unicode 标准定义了从字符到码点（整数）的映射，但直接在 Unicode 码点上训练分词器并不实际，因为词表规模过大（约 15 万个条目）且稀疏（许多字符极为罕见）。取而代之的是，我们将使用 Unicode 编码，即将 Unicode 字符转换为字节序列。Unicode 标准本身定义了三种编码：UTF-8、UTF-16 和 UTF-32，其中 UTF-8 是互联网上的主流编码（超过 98% 的网页采用）。

要将 Unicode 字符串编码为 UTF-8，可使用 Python 中的 `encode()` 函数。要访问 Python bytes 对象底层的字节值，可对其进行迭代（例如调用 `list()`）。最后，可使用 `decode()` 函数将 UTF-8 字节字符串解码为 Unicode 字符串。

```
>>> test_string = 'hello! こんにちは!'
>>> utf8_encoded = test_string.encode('utf-8')
>>> print(utf8_encoded)
b'hello! \xe3\x81\x93\xe3\x82\xe3\x93\xe3\x81\xab\xe3\x81\xa1\xe3\x81\xaf!'
>>> print(type(utf8_encoded))
<class 'bytes'>
>>> # 获取编码后字符串的字节值（0 至 255 范围内的整数）。
>>> list(utf8_encoded)

[104, 101, 108, 108, 111, 33, 32, 227, 129, 147, 227, 130, 147, 227, 129, 171, 227, 129, 161, 227, 129, 175, 33]
>>> # 一个字节不一定对应一个 Unicode 字符!
>>> print(len(test_string))

13
>>> print(len(utf8_encoded))

23
>>> print(utf8_encoded.decode('utf-8'))
hello!
こんにちは!
```

通过将 Unicode 码点转换为字节序列（例如，采用 UTF-8 编码），我们实质上是将码点序列（21 位整数，共 159,801 个有效值）转换为字节值序列（0 至 255 范围内的整数）。256 长度的字节词表更易于处理。在使用字节级分词时，我们无需担心未登录词问题，因为任何输入文本均可表示为 0 至 255 范围内整数构成的序列。

题目 (unicode2) : Unicode 编码 (3 分)

(a) 相比 UTF-16 或 UTF-32，为何更倾向于在 UTF-8 编码的字节上训练我们的分词器？对比不同编码对各类输入字符串的输出可能有助于理解。

提交要求：用一至两句话作答。

(b) 考虑以下这个（错误的）函数，其本意是将 UTF-8 字节字符串解码为 Unicode 字符串。该函数为何不正确？请提供一个导致错误结果的输入字节字符串示例。

```
def decode_utf8_bytes_to_str_wrong(bytestring: bytes):  
    return ''.join([bytes([b]).decode('utf-8') for b in bytestring])  
  
decode_utf8_bytes_to_str_wrong('hello'.encode('utf-8'))  
'hello'
```

提交要求：提供一个输入字节字符串示例，使得 `decode_utf8_bytes_to_str_wrong` 函数产生错误输出，并用一句话解释该函数为何不正确。

(c) 给出一个无法解码为任何 Unicode 字符的双字节序列。提交要求：提供一个示例，并附上一句简要说明。

2.3 子词分词

尽管字节级分词可缓解词级分词器所面临的未登录词问题，但将文本分词为字节会导致输入序列极长。这会拖慢模型训练速度：一个包含 10 个单词的句子，在词级语言模型中可能仅对应 10 个词元，但在字符级模型中却可能长达 50 个或更多词元（取决于单词长度）。处理这些更长的序列会在模型每一步中消耗更多计算资源。此外，基于字节序列的语言建模较为困难，因为更长的输入序列会在数据中引入长期依赖关系。

子词分词介于词级分词器与字节级分词器之间。注意，字节级分词器的词表包含 256 个条目（字节值范围为 0 至 255）。子词分词器则以更大的词表规模为代价，换取对输入字节序列更优的压缩效果。例如，若字节序列 `b'the'` 在原始文本训练数据中频繁出现，则为其在词表中分配一个条目，可将原本的三词元序列压缩为单个词元。

我们如何选择要加入词表的这些子词单元？R. Sennrich 等人 [3] 提出使用字节对编码（BPE；P. Gage [5]），这是一种压缩算法，通过迭代地将最频繁出现的字节对替换（合并）为一个全新的、尚未使用的索引。注意，该算法通过最大化输入序列的压缩率来向词表中添加子词词元——若某个词在输入文本中出现足够多次，它将被表示为单个子词单元。

通过 BPE 构建词表的子词分词器通常被称为 BPE 分词器。在本任务中，我们将实现一种字节级 BPE 分词器，其词表项为字节或合并后的字节序列，从而兼顾了未登录词处理能力与可控的输入序列长度。构建 BPE 分词器词表的过程称为训练 BPE 分词器。

2.4 BPE 分词器训练

BPE 分词器训练流程包含三个主要步骤。

词表初始化

分词器词表是字节字符串词元到整数 ID 的一对一映射。由于我们正在训练一个字节级 BPE 分词器，初始词表即为所有字节的集合。由于共有 256 种可能的字节值，因此初始词表大小为 256。

预分词

一旦拥有了词表，原则上你可以统计字节在文本中相邻出现的频率，并从最频繁的字节对开始逐步合并。然而，这种方法计算开销相当大，因为每次合并都需要对整个语料库进行一次完整遍历。

此外，直接在整个语料库中合并字节可能会产生仅标点符号不同的词元（例如，dog! 与 dog.）。这些词元将获得完全不同的词元 ID，尽管它们很可能具有很高的语义相似性（因为它们仅在标点符号上存在差异）。

为避免此问题，我们对语料库进行预分词。可将其理解为对语料库进行粗粒度分词，以辅助统计字符对的出现频次。例如，单词'text'可能是一个出现10次的预分词。此时，当统计字符t与e相邻出现的频次时，我们发现单词text中t与e相邻，即可将它们的计数增加10，而无需在整个语料库中逐一查找。由于我们训练的是字节级BPE模型，每个预分词均表示为UTF-8字节序列。

R. Sennrich等人[3]的原始BPE实现通过简单地按空格切分（即`s.split()`）进行预分词。这种做法仍见于基于SentencePiece的分词器中（例如Llama 1和2的分词器）。

大多数现代分词器采用基于正则表达式的预分词器，该做法源自GPT-2（A. Radford等人[6]）。我们将采用原始正则表达式的一种稍加美化形式，该形式取自

github.com/openai/tiktoken/pull/234/files:

```
> > > PAT = r'(?[sdmt]|ll|ve|re) ?\p{L}+| ?\p{N}+| ?(?:\s\p{L}\p{N})+|\s+(?!S)\s+'
```

交互式地用此预分词器对一些文本进行切分，有助于更深入理解其行为：

```
> > > # 需安装`regex`包 > > >
import regex as re
> > > re.findall(PAT, some text that i'll pre-tokenize) ['some', ' text', ' that', ' i', 'll',
' pre', '-', 'tokenize']
```

但在代码中使用时，您应使用`re.finditer()`以避免在构建预分词到其计数的映射时存储预分词。

计算BPE合并

现在，我们已将输入文本转换为预分词，并将每个预分词表示为UTF-8字节序列，便可计算BPE合并（即训练BPE分词器）。总体而言，BPE算法迭代地统计每对字节的出现频次，并识别出频次最高的字节对(A, B)；随后将该最高频字节对(A, B)的所有出现位置合并，即替换为新标记AB。该新合并标记将被加入我们的词表；因此，BPE训练后的最终词表大小等于初始词表大小（本例中为256）加上训练过程中执行的BPE合并操作次数。为提高BPE训练效率，我们不考虑跨越预分词边界的字节对。² 在计算合并时，若字节对频次相同时，需确定性地按字典序更大者优先的原则打破平局。例如，若字节对(A, B)、(A, C)、(B, ZZ)和(BA, A)均具有最高频次，则我们合并(BA, A)：

```
> > > max([(A, B), (A, C), (B, ZZ), (BA, A)]) ('BA', 'A')
```

特殊标记

通常，某些字符串（例如`<|endoftext|>`）用于编码元数据（例如文档间的边界）。在文本编码过程中，往往希望将某些字符串视为特殊标记，使其在编码时始终作为整体处理。

²请注意，原始BPE公式（R. Sennrich等人[3]）规定须包含一个词尾标记。我们在训练字节级BPE模型时不添加词尾标记，因为所有字节（包括空格与标点符号）均被纳入模型词表。由于我们显式地表示空格与标点符号，所学得的BPE合并自然能反映这些词边界。

某些字符串（例如 <|endoftext|>）应始终作为一个整体保留（即永不拆分为多个标记）。例如，序列结束字符串 <|endoftext|> 必须始终作为单个标记（即单个整数ID）保留，以便我们知晓语言模型生成何时终止。这些特殊标记必须加入词表，从而拥有对应的固定标记ID。

R. Sennrich等人[3]的算法1包含BPE分词器训练的一种低效实现（实质上遵循我们上述概述的步骤）。作为首次练习，实现并测试该函数有助于检验您的理解。

示例 (bpe_example) : BPE训练示例

以下是R. Sennrich等人[3]给出的一个风格化示例。考虑由以下文本组成的语料库：

```
low low low low low
lower lower widest widest widest
newest newest newest newest newest newest
```

且词表包含一个特殊标记 <|endoftext|>。

词表

我们以特殊标记 <|endoftext|> 及256个字节值初始化词表。

预分词

为简化问题并聚焦于合并过程，本例中我们假设预分词仅按空格切分。预分词并计数后，我们得到如下频次表。

```
{low: 5, lower: 2, widest: 3, newest: 6}
```

将其表示为dict[tuple[bytes, ...], int]形式较为便利，例如{(l,o,w): 5, ...}。请注意，在Python中即使是单个字节也属于bytes对象；Python中并无单独表示单字节的byte类型，正如其无单独表示单字符的char类型一样。

合并操作

我们首先考察每个连续字节对，并汇总其出现于各单词中的频次：{lo: 7, ow: 7, we: 8, er: 2, wi: 3, id: 3, de: 3, es: 9, st: 9, ne: 6, ew: 6}。字节对('e', 's')与('s', 't')频次并列，因此我们选择字典序更大的字节对('s', 't')。随后我们将预分词进行合并，得到{(l,o,w): 5, (l,o,w,e,r): 2, (w,i,d,e,st): 3,

(n,e,w,e,st): 6}。

第二轮中，我们发现(e, st)是最常见的字节对（计数为9），并将合并为{(l,o,w): 5, (l,o,w,e,r): 2, (w,i,d,est): 3, (n,e,w,est): 6}。依此类推，最终得到的合并序列为['s t', 'e st', 'o w', 'l ow', 'w est', 'n

e', 'ne west', 'w i', 'wi d', 'wid est', 'low e', 'lowe r']。

若执行6次合并，我们将得到['s t', 'e st', 'o w', 'l ow', 'w est', 'n e']，词表元素则为[<|endoftext|>, [...256个字节字符], st, est, ow, low, west, ne]。

借助该词表及合并规则，单词newest将被分词为[ne, west]。

2.5 BPE分词器训练实验

让我们在TinyStories数据集上训练一个字节级BPE分词器。查找/下载该数据集的说明见第1节。开始之前，我们建议您先浏览TinyStories数据集，以了解其内容。

并行化预分词

您会发现主要瓶颈在于预分词步骤。您可通过内置库multiprocessing对代码进行并行化以加速预分词。具体而言，我们建议在预分词的并行实现中，对语料库进行分块，同时确保各分块边界位于某个特殊标记的起始位置。您可以直接使用以下链接中的入门代码来确定分块边界，然后将工作分配至各进程：

https://github.com/stanford-cs336/assignment1-basics/blob/main/cs336_basics/pretokenization_example.py

这种分块方式始终有效，因为我们从不希望跨文档边界进行合并。就本作业而言，您可始终按此方式进行切分。无需担心接收一个不含 `<|endoftext|>` 的极大语料库这一边缘情况。

预分词前移除特殊标记

在使用正则表达式模式（通过`re.finditer`）执行预分词之前，您应从语料库（或在并行实现中为每个数据块）中移除所有特殊标记。请确保您按特殊标记进行切分，以防止跨特殊标记所界定的文本范围发生合并。例如，若您的语料库（或数据块）形如`[文档1] <|endoftext|> [文档2]`，则应以特殊标记 `<|endoftext|>` 为分隔符进行切分，并分别对`[文档1]`和`[文档2]`执行预分词，从而避免跨文档边界的合并。换言之，特殊标记在训练期间定义了硬性分段边界，但其本身不应参与合并计数。此操作可通过`re.split`配合以`|`连接的`special_tokens`作为分隔符完成（需谨慎使用`re.escape`，因为`|`可能出现在特殊标记中）。测试`test_train_bpe_special_tokens`将对此进行验证。

优化合并步骤

上述风格化示例中BPE训练的朴素实现之所以缓慢，是因为每次合并操作都要遍历所有字节对以找出最频繁的字节对。然而，每次合并后仅那些与被合并字节对重叠的字节对计数会发生变化。因此，可通过索引所有字节对的计数并增量更新这些计数（而非显式地遍历每一对字节以统计频率）来提升BPE训练速度。采用这种缓存机制可获得显著的速度提升，但需注意BPE训练中的合并步骤在Python中无法并行化。

低资源提示：性能分析

您应使用`cProfile`或`py-spy`等性能分析工具来识别实现中的瓶颈，并集中精力优化这些部分。

低资源提示：降级规模

我们建议您不要直接在完整的TinyStories数据集上训练分词器，而是先在数据的一个小子集——即调试数据集上进行训练。例如，您可以改用TinyStories验证集来训练分词器，该验证集包含22,000篇文档，而非212万篇。这体现了一种通用策略：尽可能降级规模以加快开发速度：

例如，使用更小的数据集、更小的模型规模等。选择调试数据集的规模或超参数配置需要仔细考量：您希望调试数据集足够大，以复现完整配置中的相同瓶颈（从而确保您所做的优化能够泛化），但又不能过大，以免运行耗时过长。

问题 (train_bpe) : BPE 分词器训练 (15 分)

交付内容：编写一个函数，该函数接收输入文本文件路径，并训练一个（字节级）BPE 分词器。您的 BPE 训练函数至少应支持以下输入参数：

输入

`input_path`: str 输入文本文件路径，用于 BPE 分词器训练。

`vocab_size`: int 一个正整数，定义最终词汇表的最大大小（包括初始字节词汇表、合并生成的词汇表项以及任何特殊标记）。`special_tokens`: list[str] 要添加至词汇表的字符串列表。在训练期间，将它们视为硬边界，防止在其跨度内进行合并，但在计算合并统计信息时不包含它们。

您的 BPE 训练函数应返回所得的词汇表和合并列表：

输出

`vocab`: dict[int, bytes] 分词器词汇表，即从整数（词汇表中的标记 ID）到字节（标记字节）的映射。

`merges`: list[tuple[bytes, bytes]] 由训练生成的 BPE 合并列表。每个列表项是一个字节元组 (<token1>, <token2>), 表示 <token1> 与 <token2> 进行了合并。合并应按创建顺序排列。

要使用我们提供的测试来验证您的 BPE 训练函数，您首先需要实现

适配器 [adapters.run_train_bpe]。然后运行 `uv run pytest tests/`

`test_train_bpe.py`。您的实现应能通过所有测试。可选地（这可能需要大量时间投入），您可以使用某些系统语言（例如 C++（考虑 `cppyy` 或 `nanobind`）或 Rust（使用 `PyO3`））实现训练方法的关键部分。若您选择这样做，请注意哪些操作需要复制，哪些操作可直接从 Python 内存读取，并确保留下构建说明，或确保仅通过 `pyproject.toml` 即可完成构建。另外请注意，GPT-2 正则表达式在大多数正则引擎中支持不佳，且在大多数支持它的引擎中也过于缓慢。我们已验证 `Oniguruma` 具有合理的速度并支持负向先行断言，但 Python 中的 `regex` 包（如果有的话）甚至更快。

问题 (train_bpe_tinystories) : 在 TinyStories 上进行 BPE 训练 (2 分)

(a) 在 TinyStories 数据集上训练一个字节级 BPE 分词器，最大词汇表大小设为 10,000。务必向词汇表中添加 TinyStories 的 <|endoftext|> 特殊标记。将所得词汇表和合并列表序列化至磁盘以便进一步检查。训练耗时和内存占用是多少？词汇表中最长的标记是什么？是否合理？

资源需求：≤30 分钟（无需 GPU），≤30 GB 内存

提示：您应能通过预分词阶段使用多进程以及以下两个事实，在 2 分钟内完成 BPE 训练：

(a) `<|endoftext|>` 标记用于分隔数据文件中的文档。

(b) `<|endoftext|>` 标记在应用 BPE 合并之前被作为特殊情况处理。交付内容：一至两句话的回答。

(b) 对您的代码进行性能分析。分词器训练过程的哪一部分耗时最长？交付内容：一至两句话的回答。

接下来，我们将尝试在 OpenWebText 数据集上训练一个字节级 BPE 分词器。与之前一样，我们建议先查看该数据集，以更好地理解其内容。

问题 (train_bpe_expts_owt)：在 OpenWebText 上进行 BPE 训练 (2 分)

(a) 在 OpenWebText 数据集上训练一个字节级 BPE 分词器，最大词汇表大小设为 32,000。将所得词汇表和合并列表序列化至磁盘以便进一步检查。词汇表中最长的标记是什么？是否合理？

资源需求：≤12 小时（无需 GPU），≤100 GB 内存；交付内容：一至两句话的回答。

(b) 比较并对比在 TinyStories 和 OpenWebText 上训练所得的分词器。

交付内容：一至两句话的回答。

2.6 BPE 分词器：编码与解码

在作业的前一部分，我们实现了在输入文本上训练 BPE 分词器以获取分词器词汇表和 BPE 合并列表的函数。现在，我们将实现一个 BPE 分词器，它加载所提供的词汇表和合并列表，并利用它们将文本编码和解码为/自标记 ID。

2.6.1 文本编码

BPE 对文本的编码过程与其训练 BPE 词汇表的方式类似。该过程包含若干主要步骤。

步骤 1：预分词。我们首先对序列进行预分词，并将每个预分词表示为 UTF-8 字节序列，这与我们在 BPE 训练中所做的一致。我们将在此预分词内将这些字节合并为词汇表元素，并独立处理每个预分词（预分词边界之间不进行合并）。步骤 2：应用合并。然后，我们采用 BPE 训练期间创建的词汇表元素合并序列，并按其创建顺序应用于我们的预分词。

示例 (bpe_encoding)：BPE 编码示例

例如，假设我们的输入字符串为 'the cat ate'，我们的词汇表为 {0: b' ', 1: b'a', 2: b'c', 3: b'e', 4: b'h', 5: b't', 6: b'th', 7: b' c', 8: b' a', 9: b'the', 10: b' at'}, 且我们学习到的合并为 [(b't', b'h'), (b' ', b'c'), (b' ', b'a'), (b'th', b'e'), (b' a',

b't']。首先，我们的预分词器会将该字符串拆分为 ['the', 'cat', 'ate']。然后，我们将逐一查看每个预分词并应用 BPE 合并。

第一个预分词 'the' 最初表示为 [b't', b'h', b'e']。查看我们的合并列表，我们识别出首个适用的合并为 (b't', b'h')，并用其将预分词转换为 [b'th', b'e']。接着，我们回到合并列表，识别出下一个适用的合并为 (b'th', b'e')，从而将预分词转换为 [b'the']。最后，再次查看合并列表，我们发现已无更多适用于该字符串的合并（因为整个预分词已被合并为单个标记），因此 BPE 合并应用完毕。对应的整数序列为 [9]。

对剩余的预分词重复此过程，我们发现预分词 'cat' 在应用 BPE 合并后表示为 [b'c', b'a', b't']，进而转化为整数序列 [7, 1, 5]。最后一个预分词 'ate' 在应用 BPE 合并后为 [b'at', b'e']，进而转化为整数序列 [10, 3]。因此，对输入字符串编码的最终结果为

[9, 7, 1, 5, 10, 3]。

特殊标记

您的分词器在对文本进行编码时，应能正确处理用户定义的特殊标记（在构建分词器时提供）。

内存考量

假设我们要对一个无法全部载入内存的大型文本文件进行分词。为高效地对该大型文件（或任何其他数据流）进行分词，我们需要将其拆分为可管理的块，并依次处理每个块，从而使内存复杂度保持恒定，而非随文本大小呈线性增长。在此过程中，我们必须确保标记不会跨越块边界，否则将得到与朴素方法（即一次性在内存中对整个序列进行分词）不同的分词结果。

2.6.2 文本解码

要将整数标记 ID 序列解码回原始文本，我们只需在词汇表中查找每个 ID 对应的条目（字节序列），将它们连接起来，然后将字节解码为 Unicode 字符串。请注意，输入的 ID 并不保证能映射为有效的 Unicode 字符串（因为用户可以输入任意整数 ID 序列）。若输入的标记 ID 无法生成有效的 Unicode 字符串，则应使用官方 Unicode 替换字符 U+FFFD 替换损坏的字节。bytes.decode 的 errors 参数控制 Unicode 解码错误的处理方式，使用 errors='replace' 可自动将损坏的数据替换为替换标记。

问题（分词器）：实现分词器（15 分）

交付内容：实现一个 Tokenizer 类，该类在给定词汇表和合并列表的情况下，将文本编码为整数 ID，并将整数 ID 解码为文本。您的分词器还应支持用户提供的特殊标记（若这些标记尚未存在于词汇表中，则将其追加至词汇表）。我们推荐采用以下接口：

3 更多关于 Unicode 替换字符的信息，请参见 [en.wikipedia.org/wiki/Specials_\(Unicode_block\)#Replacement_character](https://en.wikipedia.org/wiki/Specials_(Unicode_block)#Replacement_character)。

def __init__(self, vocab, merges, special_tokens=None) 根据给定的词汇表、合并列表和（可选的）特殊标记列表构建一个分词器词汇表、合并列表以及（可选的）特殊标记列表。该函数应接受以下参数：

```
vocab: dict[int, bytes]
merges: list[tuple[bytes, bytes]]
special_tokens: list[str] | None = None
```

def from_files(cls, vocab_filepath, merges_filepath, special_tokens=None) 类方法

该方法根据序列化的词汇表、合并列表（格式与您的 BPE 训练代码输出的格式相同）以及（可选的）特殊标记列表来构建并返回一个 Tokenizer。此方法应接受以下附加参数：

```
vocab_filepath: str
merges_filepath: str
special_tokens: list[str] | None = None
```

def encode(self, text: str) -> list[int]: 将输入文本编码为一串词元 ID。

def encode_iterable(self, iterable: Iterable[str]) -> Iterator[int]: 给定一个字符串可迭代对象，字符串（例如 Python 文件句柄），返回一个惰性生成器，逐个产出词元 ID。这对内存受限的大文件分词至关重要，因为这类文件无法直接加载进内存。

def decode(self, ids: list[int]) -> str: 将一串词元 ID 解码为文本。

要针对我们提供的测试用例检验你的 Tokenizer，你首先需实现测试

适配器位于 [adapters.get_tokenizer]。然后运行 `uv run pytest tests/test_tokenizer.py`。你的实现应能通过全部测试。

2.7 实验

题目 (tokenizer_experiments)：分词器实验 (4 分)

(a) 从 TinyStories 和 OpenWebText 中各采样 10 篇文档。使用你此前训练好的 TinyStories 和 OpenWebText 分词器（词汇量分别为 10K 和 32K），将这些采样的文档编码为整数 ID。每个分词器的压缩比（字节/词元）是多少？

交付内容：一至两句话的回答。

(b) 若用 TinyStories 分词器对你的 OpenWebText 样本进行分词，会发生什么？比较压缩比和/或定性描述发生的情况。交付内容：一至两句话的回答。

(c) 估算你的分词器吞吐量（例如，单位为字节/秒）。对 Pile 数据集（825GB 文本）进行分词需要多长时间？

交付内容：一至两句话的回答。

(d) 使用你的 TinyStories 和 OpenWebText 分词器，将各自对应的训练集和开发集编码为整数词元 ID 序列。稍后我们将用此数据训练我们的

语言模型。我们建议将词元 ID 序列序列化为 dtype 为 uint16 的 NumPy 数组。为何 uint16 是合适的选择？

交付内容：一至两句话的回答。

3 Transformer 语言模型架构

语言模型接收一个批处理的整数词元 ID 序列作为输入（即形状为 $(\text{batch_size}, \text{sequence_length})$ 的 `torch.Tensor`），并返回一个（批处理的）词汇表上的归一化概率分布（即形状为 $(\text{batch_size}, \text{sequence_length}, \text{vocab_size})$ 的 PyTorch Tensor），其中对每个输入词元，预测的分布对应其后一个词元。训练语言模型时，我们利用这些下一个词元的预测结果计算实际下一个词元与预测下一个词元之间的交叉熵损失；推理阶段生成文本时，我们取最终时间步（即序列中的最后一个项）所预测的下一个词元分布来生成序列中的下一个词元（例如，选取概率最高的词元、从分布中采样等），将生成的词元加入输入序列，并重复该过程。

在本作业的这一部分，你将从零开始构建该 Transformer 语言模型。我们将先给出模型的高层描述，再逐步详述各个组成部分。

3.1 Transformer 语言模型

给定一个词元 ID 序列，Transformer 语言模型首先使用输入嵌入将词元 ID 转换为稠密向量，然后将这些嵌入后的词元依次通过 num_layers 个 Transformer 块，最后应用一个可学习的线性投影（即输出嵌入或语言模型头），以生成预测的下一个词元的 logits。图 1 给出了该过程的示意图。

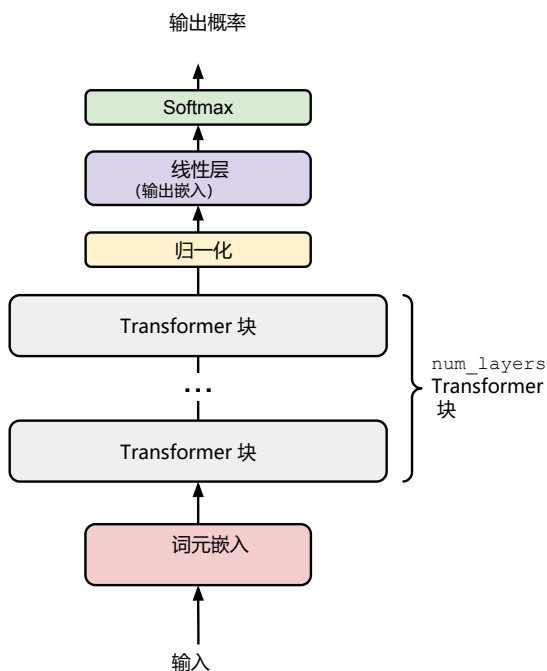


图 1：我们所用 Transformer 语言模型的概览。

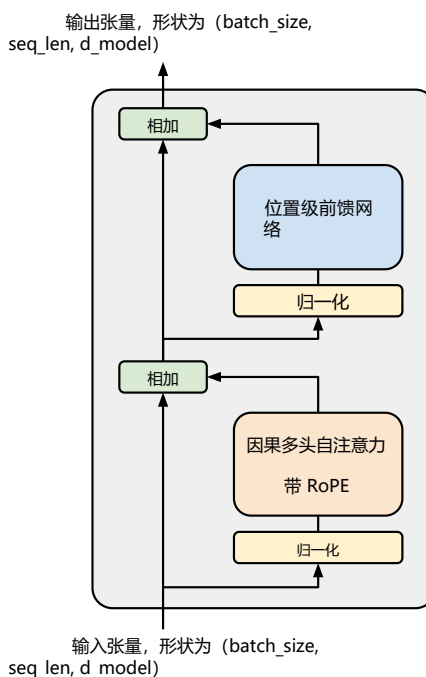


图 2：预归一化的 Transformer 块。

词元嵌入

在最开始的步骤中，Transformer 将（批处理的）词元 ID 序列嵌入为一组向量序列，其中每个向量包含该词元身份的信息（图 1 中的红色模块）。 

更具体地说，给定一个词元 ID 序列，Transformer 语言模型使用词元嵌入层生成一个向量序列。每个嵌入层接收一个形状为 $(batch_size, sequence_length)$ 的整数张量，并输出一个形状为 $(batch_size,$

序列长度, $d_model)$ 。

预归一化 Transformer 块

嵌入之后，激活值由若干结构完全相同的神经网络层进行处理。标准的仅解码器 Transformer 语言模型由 num_layers 个相同结构的层（通常称为 Transformer 块）构成。每个 Transformer 块接收形状为（批量大小，

序列长度, $d_model)$ 并返回形状为（批量大小，序列长度, $d_model)$ 的输出。

每个块均在序列上聚合信息（通过自注意力机制），并对其进行非线性变换（通过前馈层）。

在经过 num_layers 个 Transformer 块后，我们将取最终激活值并将其转化为词汇表上的分布。

我们将实现预归一化 Transformer 块（详见第 3.4 节），该块还需在最终 Transformer 块后使用层归一化（详见下文），以确保其输出得到恰当缩放。

在此归一化之后，我们将使用标准的可学习线性变换，将 Transformer 块的输出转换为预测的下一个词元 logits（参见，例如，A. Radford 等人 [7] 的公式 2）。 

3.2 备注：批处理、Einsum 与高效计算

在整个 Transformer 架构中，我们将对大量类批处理输入执行相同的计算。以下是几个例子：

- 批处理中的元素：我们对每个批处理元素应用相同的 Transformer 前向操作。
- 序列长度：RMSNorm 和前馈网络等按位置操作在序列的每个位置上均以相同方式执行。
- 注意力头：在多头注意力操作中，注意力操作会在各个注意力头上进行批处理。

拥有一种既充分利用 GPU、又易于阅读和理解的、执行此类操作的人体工学方式是很有用的。许多 PyTorch 操作可以接受张量开头多余的类批处理维度，并高效地在这些维度上重复/广播操作。

例如，假设我们正在执行一个按位置进行的批处理操作。我们有一个形状为（批量大小，序列长度, $d_model)$ 的数据张量，并希望对该张量与一个形状为 (d_model, d_model) 的矩阵 A 进行批处理向量-矩阵乘法。此时， $D @ A$ 将执行批处理矩阵乘法，这是 PyTorch 中的一个高效原语，其中（批量大小，序列长度）维度被当作批处理维度。

因此，假定您的函数可能被传入额外的类批处理维度，并将这些维度保持在 PyTorch 形状的开头，是有益的。为了使张量能够以这种方式进行批处理，可能需要通过多次 view、reshape 和 transpose 来调整其形状。这可能略显繁琐，而且往往难以理解代码的实际意图以及张量的形状。

一种更符合人体工学的方案是，在 `torch.einsum` 中使用 `einsum` 表示法，或者采用框架无关的库如 `einops` 或 `einx`。这两个关键操作是：`einsum`，可用于对输入张量的任意维度进行张量缩并；`rearrange`，可用于对任意维度进行重排序、拼接与拆分。事实上，机器学习中的绝大多数操作都是维度操作与张量缩并的某种组合，偶尔辅以（通常是逐点）非线性函数。这意味着，使用 `einsum` 表示法可使您的大量代码更具可读性和灵活性。

我们强烈建议本课程中学习并使用 `einsum` 表示法。此前未接触过 `einsum` 表示法的学生应使用 `einops`（文档在此处），而已经熟悉 `einops` 的学生则应学习更通用的 `einx`（在此处）。⁴ 这两个包已在我们提供的环境中预先安装。

此处提供一些 `einsum` 表示法的使用示例，作为 `einops` 文档的补充内容，您应首先阅读其文档。

示例 (einstein_example1)：使用 `einops.einsum` 的批处理矩阵乘法

```
import torch
from einops import rearrange, einsum

## 基础实现: Y = D @ A.T

# 很难判断输入和输出的形状及其含义。
# D 和 A 可以具有哪些形状？其中是否存在意外行为？

## Einsum 具有自解释性和鲁棒性 # D
                                     A →
Y = einsum(D, A, 批量大小 序列长度 d_in, d_out d_in → 批量大小 序列长度 d_out)

## 或者，一个批处理版本，其中 D 可以拥有任意数量的前置维度，而 A 的维度则受到约束。□ Y = einsum(D, A, ...
d_in, d_out d_in → ... d_out)
```

示例 (einstein_example2)：使用 `einops.rearrange` 的广播操作

我们有一批图像，且希望对每张图像基于某个缩放因子生成 10 个明暗程度不同的版本：

```
images = torch.randn(64, 128, 128, 3) # (批量大小, 高度, 宽度, 通道数) □ dim_by =
torch.linspace(start=0.0, end=1.0, steps=10)

## 重塑并相乘
dim_value = rearrange(dim_by, 维度值 → 1 维度值 1 1 1)
images_rearr = rearrange(images, b height width channel → b 1 height width channel) □ dimmed_images =
images_rearr * dim_value

## 或一步完成:
dimmed_images = einsum(
    图像, 缩放因子
    批量大小 高度 宽度 通道数, 维度值 → 批量大小 维度值 高度 宽度 通道数
)
```

值得注意的是，尽管 `einops` 拥有广泛的支持，`einx` 尚未经过同等程度的实战检验。如果您在使用 `einx` 时遇到任何限制或缺陷，完全可以回退到结合更基础的 PyTorch 操作来使用 `einops`。

示例 (einstein_example3) : 使用 einops.rearrange 进行像素混合

假设我们有一批图像，表示为形状为（批量大小，高度，宽度，通道数）的张量，我们希望对图像的所有像素执行线性变换，但该变换需在每个通道上独立进行。我们的线性变换由一个形状为（高度 × 宽度，高度 × 宽度）的矩阵表示。

```
channels_last = torch.randn(64, 32, 32, 3) # （批量大小，高度，宽度，通道数） ⊠ B =  
torch.randn(32*32, 32*32)
```

使用 einops.rearrange 对图像张量进行重排，以实现所有像素间的混合：

```
channels_last_flat = channels_last.view(  
    -1, channels_last.size(1) * channels_last.size(2), channels_last.size(3)  
)  
channels_first_flat = channels_last_flat.transpose(1, 2)  
channels_first_flat_transformed = channels_first_flat @ B.T  
channels_last_flat_transformed = channels_first_flat_transformed.transpose(1, 2)  
channels_last_transformed = channels_last_flat_transformed.view(*channels_last.shape)
```

相反，使用 einops：

高度 = 宽度 = 32

Rearrange 替代了繁琐的 torch view + transpose。通道在最前

```
(channels_first) = rearrange(  
    通道在最后 (channels_last)  
    批量大小 高度 宽度 通道数 → 批量大小 通道数 (高度 宽度)  
)  
channels_first_transformed = einsum(  
    channels_first, B,  
    batch channel pixel_in, pixel_out pixel_in -> batch channel pixel_out  
)  
channels_last_transformed = rearrange(  
    channels_first_transformed,  
    batch channel (height width) -> batch height width channel, height=height,  
    width=width  
)
```

或者，如果您愿意尝试更激进的方式：一次性使用 einx.dot (einx 中等效于 einops.einsum 的函数)

```
height = width = 32  
channels_last_transformed = einx.dot(  
    batch row_in col_in channel, (row_out col_out) (row_in col_in) -> batch row_out  
    col_out channel,  
    channels_last, B,  
    col_in=width, col_out=width  
)
```

此处的第一种实现可通过在前后添加注释来说明输入与输出形状，但这显得笨拙且易出错。而使用 einsum 记号时，文档即实现！

einsum 记号可处理任意输入批处理维度，同时还具备关键优势：自解释性。在代码中，使用 einsum 记号能更清晰地表明输入与输出张量的相关形状。

使用 einsum 记号的实现。对于其余张量，您可考虑使用 Tensor 类型提示，例如借助 jaxtyping 库（不专属于 JAX）。

我们将在作业 2 中进一步讨论使用 einsum 记号的性能影响，但目前请记住：它们几乎总是优于替代方案！

3.2.1 数学记号与内存布局

许多机器学习论文采用行向量记号，这与 NumPy 和 PyTorch 默认使用的行优先内存布局相契合。使用行向量时，线性变换形式为

$$y = T x, \quad (1)$$

其中 $\in \text{out} \times \text{in}$ 为行优先存储， $\in 1 \times \text{in}$ 为行向量。请注意，这使得我们可通过扩展的最外层维度来批处理输入，即可用矩阵输入 $\in \text{batch} \times \text{in}$ 替代向量输入。在线性代数中，列向量更为常见，此时线性变换形式为

$$y = T^T x, \quad (2)$$

给定行优先存储的 $\in \text{out} \times \text{in}$ 和列向量 $\in \text{in}$ 。在此设定下对输入进行批处理时，批维度需置于的末尾，因此需替换为矩阵 $\in \text{in} \times \text{batch}$ 。本作业中数学符号主要采用列向量表示法，因为数学惯例普遍如此。您需留意：若想使用标准矩阵乘法符号，则需按式 (1) 中的行向量惯例对矩阵进行转置操作，因为 PyTorch 默认采用行优先内存布局。若您使用 einsum 进行线性代数运算，只要正确标注轴名，这便不会成为问题。顺便一提，其他语言/线性代数库（如 Matlab、Julia 和 Fortran）均采用列优先内存布局，意味着批维度位于末尾；而 Python 及相关包则遵循 C 标准的行优先内存布局。

3.3 基础构建模块：线性与嵌入模块

3.3.1 参数初始化

高效训练神经网络通常需要谨慎地初始化模型参数——不良的初始化可能导致梯度消失或爆炸等不良行为。预归一化 Transformer 对初始化异常鲁棒，但初始化仍可能显著影响训练速度与收敛性。由于本作业已较长，我们将细节留待作业 3 讲解，此处仅提供一些适用于大多数情况的近似初始化方法。目前请使用：

- 线性权重：($\in \text{out} \times \text{in}$) 截断区间为 $[-3, 3]$ 。
- 嵌入：($\in \text{out} \times 1$)，截断区间为 $[-3, 3]$
- RMSNorm：

您应使用 `torch.nn.init.trunc_normal_` 初始化截断正态分布权重。

3.3.2 线性模块

线性层是 Transformer 和通用神经网络的基本构成模块。首先，您将实现自己的 Linear 类，该类继承自 `torch.nn.Module` 并执行线性变换：

= 。

(3)

注意，我们未包含偏置项，以符合大多数现代大语言模型的做法。

问题（线性）：实现线性模块（1 分）

交付成果：实现一个继承自 `torch.nn.Module` 的 `Linear` 类，执行线性变换。您的实现应遵循 PyTorch 内置 `nn.Linear` 模块的接口，但不包含 `bias` 参数或参数。我们推荐以下接口：

`def __init__(self, in_features, out_features, device=None, dtype=None)` 构造一个线性变换模块。该函数应接受以下参数：`in_features: int` 输入的最终维度

`out_features: int` 输出的最终维度

`device: torch.device | None = None` 参数所存放的设备 `dtype: torch.dtype | None = None` 参数的数据类型

`def forward(self, x: torch.Tensor) -> torch.Tensor` 对输入应用线性变换
输入。请确

保：

- 继承 `nn.Module`
- 调用父类构造函数
- 将参数构造并存储为 `Parameter`（而非 `Tensor`），并将其放入 `nn.Parameter` 中
- 当然，不得使用 `nn.Linear` 或 `nn.functional.linear`

初始化时，请沿用上述设置，并使用 `torch.nn.init.trunc_normal_` 初始化权重。

为测试您的 `Linear` 模块，请在 `[adapters.run_linear]` 处实现测试适配器。该适配器应将给定权重加载到您的 `Linear` 模块中。您可以使用 `Module.load_state_dict` 来此目的。然后运行 `uv run pytest -k test_linear`。

3.3.3 嵌入模块

如上所述，Transformer 的第一层是嵌入层，它将整数 token ID 映射到维度为 `d_model` 的向量空间中。我们将实现一个自定义的 `Embedding` 类，该类继承自 `torch.nn.Module`（因此您不应使用 `nn.Embedding`）。其 `forward` 方法应通过索引形状为 `(vocab_size, d_model)` 的嵌入矩阵，为每个 token ID 选取对应的嵌入向量，使用形状为 `(batch_size, sequence_length)` 的 token ID 的 `torch.LongTensor`。

问题（嵌入）：实现嵌入模块（1 分）

交付成果：实现一个继承自 `torch.nn.Module` 的 `Embedding` 类，执行嵌入查找。您的实现应遵循 PyTorch 内置 `nn.Embedding` 模块的接口。我们推荐以下接口：

`def __init__(self, num_embeddings, embedding_dim, device=None, dtype=None)` 构造一个嵌入模块。该函数应接受以下参数：`num_embeddings: int` 词汇表大小

`embedding_dim: int` 嵌入向量维度，即模型维度

`device: torch.device | None = None` 参数存储设备 `dtype: torch.dtype | None = None`

参数数据类型

`def forward(self, token_ids: torch.Tensor) -> torch.Tensor` 查找对应 token ID 的嵌入向量，以获取对应 token ID 的

嵌入向量。请确保：

- 继承 `nn.Module`
- 调用父类构造函数
- 将嵌入矩阵初始化为 `nn.Parameter`
- 将嵌入矩阵的 `d_model` 设为最终维度
- 当然，不要使用 `nn.Embedding` 或 `nn.functional.embedding`

同样，请使用上述设置进行初始化，并使用 `torch.nn.init.trunc_normal_` 初始化权重。

为测试您的实现，请在 `[adapters.run_embedding]` 处实现测试适配器，然后运行 `uv run pytest -k test_embedding`。

3.4 预归一化 Transformer 块

每个 Transformer 块包含两个子层：多头自注意力机制和逐位置前馈网络 ([A. Vaswani 等人, 2017], 第 3.1 节)。

在原始 Transformer 论文中，模型在每个子层周围使用残差连接，随后进行层归一化。这种架构通常被称为后归一化 Transformer，因为层归一化应用于子层输出。然而，大量研究发现，将层归一化从每个子层的输出移至输入（并在最终 Transformer 块后额外添加一层归一化）可提升 Transformer 的训练稳定性 [T. Q. Nguyen 等人, 2019; R. Xiong 等人, 2020] —— 图 2 展示了这种预归一化 Transformer 块的可视化表示。每个 Transformer 块子层的输出随后通过残差连接加到子层输入上 (A. Vaswani 等人 [8], 第 5.4 节)。预归一化的直观理解是：存在一条干净的、未经任何归一化的残差通路，从输入嵌入直接通往 Transformer 的最终输出，这据称可改善梯度流动。如今，这种预归一化 Transformer 已成为语言模型的标准（例如 GPT-3、LLaMA、PaLM 等），因此我们将实现此变体。接下来，我们将逐步实现预归一化 Transformer 块的各个组件。

3.4.1 均方根层归一化

原始 Transformer 实现 (A. Vaswani 等人 [8]) 使用层归一化 [J. L. Ba 等人, 2016] 用于归一化激活值。遵循 H. Touvron 等人 [12] 的做法，我们将采用均方根层归一化 (RMSNorm; B. Zhang 等人 [13], 公式 4) 进行层归一化。给定一个属于 model 的激活向量，RMSNorm 将按如下方式对每个激活值 进行重新缩放：

$$\text{RMSNorm}() = \frac{\text{RMS}()}{\quad}, \quad (4)$$

其中 $\text{RMS}() = \sqrt{1 + \frac{1}{d_{\text{model}}} \sum_{\text{模型}} 1}$ 。此处， γ 是一个可学习的增益参数（共有 d_{model} 个此类参数，参数总数），而 ϵ 是一个常设为 $1e-5$ 的超参数。

您应将输入向上转换为 `torch.float32`，以防对输入进行平方时发生溢出。总体而言，您的 `forward` 方法应如下所示：

```
in_dtype = x.dtype
x = x.to(torch.float32)

# 此处编写执行 RMSNorm 的代码
...
result = ...

# 以原始数据类型返回结果 return result.to(in_dtype)
```

题目 (rmsnorm)：均方根层归一化 (1 分)

交付成果：将 `RMSNorm` 实现为 `torch.nn.Module`。我们推荐以下接口：

`def __init__(self, d_model: int, eps: float = 1e-5, device=None, dtype=None)` 构造 `RMSNorm` 模块。该函数应接受以下参数：`d_model: int` 模型隐藏维度

`eps: float = 1e-5` 数值稳定性用的 `epsilon` 值

`device: torch.device | None = None` 参数存储设备 `dtype: torch.dtype | None = None`

参数数据类型

`def forward(self, x: torch.Tensor) -> torch.Tensor` 处理形状为

`(batch_size, sequence_length, d_model)`，并返回相同形状的张量。

注意：请记住，如上所述，在执行归一化之前，先将输入向上转换为 `torch.float32`（之后再向下转换回原始数据类型）。

为测试您的实现，请在 `[adapters.run_rmsnorm]` 处实现测试适配器，然后运行 `uv run pytest -k test_rmsnorm`。

3.4.2 逐位置前馈网络

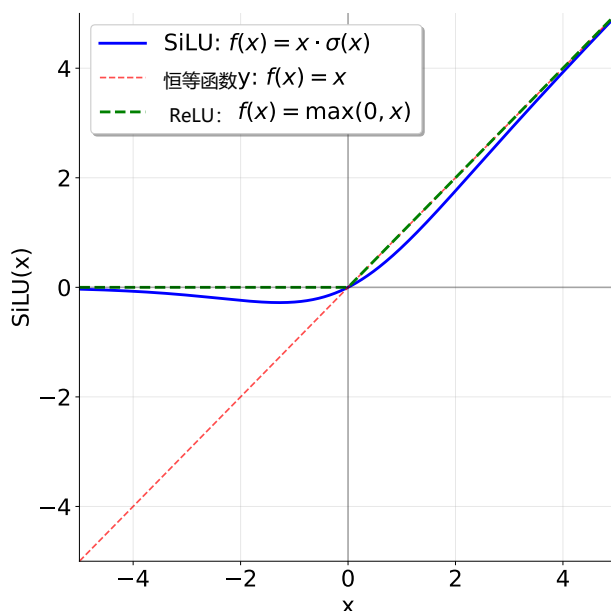


图 3: SiLU (又称 Swish) 与 ReLU 激活函数对比。

在原始 Transformer 论文 (A. Vaswani 等 [8] 第 3.3 节) 中, Transformer 的前馈网络由两个线性变换构成, 中间夹着一个 ReLU 激活函数 ($\text{ReLU}(x) = \max(0, x)$)。在该原始架构中, 内部前馈层的维度通常是输入维度的 4 倍。

然而, 现代语言模型相较于原始设计主要引入了两项改动: 采用另一种激活函数, 并引入门控机制。具体而言, 我们将实现被 Llama 3 [A. Grattafiori 等, 2024] 和 Qwen 2.5 [A. Yang 等, 2024] 等大语言模型所采用的 SwiGLU 激活函数, 该函数将 SiLU (常称 Swish) 激活与一种名为门控线性单元 (GLU) 的门控机制相结合。[我们还将省略线性层中有时使用的偏置项, 这遵循 PaLM [A. Chowdhery 等, 2022] 和 LLaMA 等大多数现代大语言模型的做法。

[H. Touvron 等, 2023]。

SiLU 或 Swish 激活函数 [D. Hendrycks 等, 2016; S. Elfwing 等, 2017] 定义如下:

$$\text{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}} \quad (5)$$

如图 3 所示, SiLU 激活函数与 ReLU 激活函数相似, 但在零点处平滑。

门控线性单元 (GLU) 最初由 Y. N. Dauphin 等人 [19] 定义为: 一个经 sigmoid 函数变换的线性变换与另一个线性变换的逐元素乘积:

$$\text{GLU}(1,2) = (1) \odot 2 \quad (6)$$

其中 $\odot$ 表示逐元素乘法。门控线性单元 (GLU) 旨在通过为梯度提供一条线性通路, 同时保留非线性能力, 从而缓解深度架构中的梯度消失问题。

将 SiLU/Swish 与 GLU 结合，我们得到 SwiGLU，它将用于我们的前馈网络：

$$\text{FFN}() = \text{SwiGLU}(1, 2, 3) = 2(\text{SiLU}(1) \odot 3) \quad (7)$$

其中 $1 \in \text{model}$, $1, 3 \in \text{ff} \times \text{model}$, $2 \in \text{model} \times \text{ff}$, 且按惯例, $\text{ff} = 8 \times \text{模型}$ 。针对具体实现时，为提升硬件效率，可将该值四舍五入至最接近的 64 的倍数。N. Shazeer [20] 首次提出将 SiLU/Swish 激活函数与 GLU 结合，并通过实验表明 SwiGLU 在语言建模任务上优于 ReLU 和无门控的 SiLU 等基线方法。在后续作业中，你将比较 SwiGLU 与 SiLU。尽管我们已提及这些组件的一些启发式理由（相关论文也提供了更多支持性证据），但保持实证视角仍是明智的：Shazeer 论文中一句如今广为流传的话是

我们不解释这些架构为何似乎有效；我们将它们的成功归因于神恩眷顾。

题目 (positionwise_feedforward)：实现逐位置前馈网络 (2 分)

交付内容：实现 SwiGLU 前馈网络，该网络由 SiLU 激活函数和门控线性单元 (GLU) 组成。

注意：在此特定情况下，为保证数值稳定性，你可在实现中自由使用 torch.sigmoid。

你应将 ff 设为约 $8 \times \text{模型}$ ，同时确保确保内部前馈层的维度是 64 的倍数，以充分利用硬件性能。为使用我们提供的测试用例验证你的实现，你需要实现
在 [adapters.run_swiglu] 处测试适配器。然后运行 `uv run pytest -k test_swiglu` 来测试你的实现。

3.4.3 相对位置嵌入

为将位置信息注入模型，我们将实现旋转位置嵌入 (Rotary Position Embeddings)

[J. Su 等, 2021]，通常称为 RoPE。对于位于位置 $i = () \in \dots$ 的查询标记 $() = () \in \dots$ ，我们将应用一个成对旋转矩阵，从而得到 $'() = () = ()$ 。此处，将对嵌入元素 $()$ 的成对分量进行旋转

$2-1:2$ 将嵌入元素 $()$ 视为二维向量，并以角度 θ 进行旋转， $\theta = \Theta(2-2)/$ ，其中 $\Theta \in \{1, \dots, /2\}$ ，且存在某个 Θ 为常数。因此，我们可以将 Θ 视为一个大小为 $\times$ 的分块对角矩阵，其分块为 Θ ，其中 $\Theta \in \{1, \dots, \}$ ，其中

$$\Theta = \begin{pmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{pmatrix} \quad (8)$$

因此我们得到完整的旋转矩阵

$$= \begin{pmatrix} 0 & 0 & \dots & 0 \\ 0 & 2 & 0 & \dots & 0 \\ 0 & 0 & 3 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & \end{pmatrix} y_r \quad (9)$$

其中 0 表示 2×2 的零矩阵。虽然可以构造完整的 $\times$ 矩阵，但一种更优的解法应利用该矩阵的性质，以更高效的方式实现变换。由于我们只关心给定序列内 token 的相对旋转，因此可在不同层及不同批次间复用 $\cos()$ 和 $\sin()$ 的计算结果。如需优化，可让所有层共用单个 RoPE 模块，并为其配备一个二维预计算的 $\sin/\cos$ 缓冲区。

在初始化期间使用 `self.register_buffer(persistent=False)` 创建的值，而非 `nn.Parameter`

（因为我们不希望学习这些固定的余弦和正弦值）。我们此前对 `()` 所做的完全相同的旋转过程，随后也应用于 `()`，按对应的 进行旋转。请注意，该层不含任何可学习参数。

题目 (rope) : 实现 RoPE (2 分)

交付内容：实现一个 `RotaryPositionalEmbedding` 类，对输入张量应用 RoPE。

推荐的接口如下：

`def __init__(self, theta: float, d_k: int, max_seq_len: int, device=None)` 构造

RoPE 模块，并在需要时创建缓冲区。theta:

float RoPE 的 Θ 值

`d_k`: int 查询向量和键向量的维度

`max_seq_len`: int 输入的最大序列长度

`device`: torch.device | None = None 缓冲区所存放的设备

`def forward(self, x: torch.Tensor, token_positions: torch.Tensor) -> torch.Tensor` 处理

一个形状为 `(..., seq_len, d_k)` 的输入张量，并返回相同形状的张量。注意，你需要支持任意数量批维度的。你应假设 `token` 位置是一个形状为 `(..., seq_len)` 的张量，用于指定在序列维度上的 `token` 位置。

你应该使用 `token` 位置沿序列维度切片你的（可能已预计算的）`cos` 和 `sin` 张量。

为测试你的实现，请完成 `[adapters.run_rope]`，并确保它能通过 `uv run pytest -k test_rope`。

3.4.4 缩放点积注意力

我们现在将按照 A. Vaswani 等人 [8] (第 3.2.1 节) 所述实现缩放点积注意力 (scaled dot-product attention)。作为预备步骤，Attention 操作的定义将用到 `softmax`——这是一种将未归一化的分数向量转换为归一化分布的操作：

$$\text{softmax}() = \frac{\exp() \sum_{=1}}{\exp()} \quad (10)$$

注意，`exp()` 在输入值较大时可能变为无穷大（此时，无穷大 = 非数字）。我们可通过观察以下事实来避免此问题：`softmax` 操作对所有输入加上任意常数 具有不变性。我们可以利用这一性质提升数值稳定性——通常做法是，从 的所有元素中减去其最大值，使新的最大值变为 0。你接下来将实现 `softmax`，并使用该技巧保证数值稳定性。

题目 (`softmax`)：实现 `softmax` (1 分)

交付内容：编写一个函数，对张量执行 `softmax` 操作。该函数应接受两个参数：一个张量和一个维度，并在输入张量的第 - 维上应用 `softmax`。输出张量的形状应与输入张量相同，但其第 - 维现在将具有归一化的概率分布。为避免数值稳定性问题，请采用技巧：从第 - 维的所有元素中减去该维的最大值。

为测试你的实现，请完成 `[adapters.run_softmax]`，并确保它能通过 `uv run pytest -k test_softmax_matches_pytorch`。

我们现在可以如下数学地定义 `Attention` 操作：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (11)$$

其中 $Q \in \mathbb{R}^{n \times d_k}$ 、 $K \in \mathbb{R}^{m \times d_k}$ 、 $V \in \mathbb{R}^{m \times d_v}$ 。此处 n 、 m 和 d_k 均为该操作的输入——注意它们并非可学习参数。

掩码：有时需要对注意力操作的输出进行掩码。掩码应具有形状 $\in \{\text{True}, \text{False}\} \times$ ，该布尔矩阵的每一行表示查询 应关注哪些键。通常（且略显混淆地），位置 (i, j) 处值为 `True` 表示查询 确实关注键 j ，而值为 `False` 则表示查询 不关注键 j 。换言之，信息仅在值为 `True` 的 (i, j) 对之间流动。例如，考虑一个 1×3 的掩码矩阵 `[[True, True, False]]`，该单个查询向量仅关注前两个键。从计算角度看，使用掩码比在子序列上计算注意力要高效得多；我们可通过将预`softmax`值 (T/V) 中掩码值为 `False` 的项设为 $-\infty$ 来实现掩码效果。

题目 (`scaled_dot_product_attention`)：实现缩放点积注意力 (5分)

交付物：实现缩放点积注意力函数。你的实现应能处理形状为 `(batch_size, ..., seq_len, d_k)` 的键和查询，以及形状为 `(batch_size, ..., seq_len, d_v)` 的值。
`(batch_size, ..., seq_len, d_v)`，其中 ... 表示任意数量的其他类批处理维度（若提供）。实现应返回形状为 `(batch_size, ..., seq_len, d_v)` 的输出。参见第3.2节关于类批处理维度的讨论。
 你的实现还应支持可选的用户提供的布尔掩码，其形状为 `(seq_len, seq_len)`。掩码值为 `True` 的位置对应的注意力概率应整体归一化为1，而掩码值为 `False` 的位置对应的注意力概率应为零。

为用我们提供的测试验证你的实现，你需要实现测试

适配器 `[adapters.run_scaled_dot_product_attention]`。 `uv run pytest -k`

`test_scaled_dot_product_attention` 测试你的实现在三维输入张量上的表现，而
`uv run pytest -k test_4d_scaled_dot_product_attention` 测试你的实现在四维输入张量上的表现。
顺序输入张量。

3.4.5 因果多头自注意力

我们将按照A. Vaswani等人[8]第3.2.2节所述实现多头自注意力。回顾可知，多头注意力操作在数学上定义如下：

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \quad (12)$$

$$\text{for head} = \text{Attention}(\mathbf{Q}_i, \mathbf{K}_i, \mathbf{V}_i) \quad (13)$$

其中， $\mathbf{Q}_i, \mathbf{K}_i, \mathbf{V}_i$ 分别为嵌入维度中尺寸为 d_k 或 d_v 的第 i 个切片（ $i \in \{1, \dots, h\}$ ），分别对应 $\mathbf{Q}, \mathbf{K}$ 和 $\mathbf{V}$ 。其中Attention为第3.4.4节所定义的缩放点积注意力操作。由此可构造多头自注意力操作：

$$\text{MultiHeadSelfAttention}() = \text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \quad (14)$$

此处可学习参数为 $d_k \times \text{model}$ 、 $d_k \times \text{model}$ 、 $d_v \times \text{model}$ 和 $\text{model} \times \text{model}$ 。由于在多头注意力操作中， $\mathbf{Q}, \mathbf{K}$ 和 $\mathbf{V}$ 被切片，我们可以认为 $\mathbf{Q}, \mathbf{K}$ 和 $\mathbf{V}$ 在输出维度上按每个头分开。当你完成此步骤后，应总共仅需三次矩阵乘法即可完成键、值和查询的投影。

因果掩码

你的实现应防止模型关注序列中的未来词元。换言之，若模型接收一个词元序列

$x_1, \dots, x_n$ ，并希望计算下一个词预测

对于前缀 $x_1, \dots, x_t$ （其中 $t < n$ ），模型不应能够访问（即参与注意力计算）位置 $t+1, \dots, n$ 处的词元表示，因为在推理生成文本时，模型无法获得这些后续词元（且这些未来词元会泄露真实下一个词的身份信息，从而使得语言建模预训练目标变得平凡）。对于输入词元序列 $x_1, \dots, x_n$ ，我们可以朴素地通过运行多头自注意力 n 次（对应序列中 n 个唯一前缀）来防止访问未来词元。相反，我们将使用因果注意力掩码，该掩码允许词元 x_t 仅关注序列中位置 $\leq t$ 的所有位置。你可以使用 `torch.triu` 或广播索引比较来构建该掩码，并应充分利用第3.4.4节中已支持注意力掩码的缩放点积注意力实现。

应用RoPE

RoPE应应用于查询向量和键向量，但不应用于值向量。此外，头维度应被视作批处理维度，因为在多头注意力中，每个头的注意力是独立应用的。这意味着，对每个头的查询向量和键向量，应精确应用相同的RoPE旋转。

题目（`multihead_self_attention`）：实现因果多头自注意力（5分）

5作为进阶目标，尝试将键、查询和值投影合并为单个权重矩阵，从而只需一次矩阵乘法。

交付物：将因果多头自注意力实现为torch.nn.Module。你的实现至少应接受以下参数：

d_model: int Transformer块输入的维度。num_heads: int 多头

自注意力中使用的头数。根据A. Vaswani等人的工作[8]，设 $h = \frac{d_{\text{model}}}{\text{num_heads}}$

model $\frac{1}{h}$ 。为用我们的提供的测试，请在 [adapters.run_multihead_self_attention] 处实现测试适配器。然后，运行 `uv run pytest -k test_multihead_self_attention` 来测试你的实现。

3.5 完整的Transformer语言模型

我们先来构建Transformer块（参考图2将很有帮助）。一个Transformer块包含两个子层：一个用于多头自注意力，另一个用于SwiGLU前馈网络。在每个子层中，我们首先执行RMSNorm，然后执行主操作（MHA/FF），最后加入残差连接。

具体而言，Transformer块的前半部分（即第一个子层）应实现如下更新，以从输入 得到输出 。

$$= + \text{MultiHeadSelfAttention}(\text{RMSNorm}())。 \quad (15)$$

题目 (transformer_block) : 实现Transformer块 (3分)

按照第3.4节所述并如图2所示，实现预归一化Transformer块。你的Transformer块至少应接受以下参数。

d_model: int Transformer块输入的维度。num_heads: int 多头自

注意力中使用的头数。d_ff: int 位置前馈网络内部层的维度。

为测试你的实现，请实现适配器 [adapters.run_transformer_block]。然后运行 `uv run pytest -k test_transformer_block` 来测试你的实现。

交付物：能通过所提供测试的Transformer块代码。

现在我们依照图1中的高层结构图将各模块组合起来。请遵循第3.1.0.1节中对嵌入层的描述，将嵌入结果输入num_layers个Transformer块，再将输出送入最终的层归一化和LM头，从而得到词汇表上的未归一化分布（即logits）。

题目 (transformer_lm) : 实现Transformer语言模型 (3分)

现在将所有部分整合起来！按照第3.1节所述并如图1所示，实现Transformer语言模型。你的实现至少应接受前述Transformer块的所有构造参数，以及以下附加参数：

vocab_size: int 词表大小，用于确定词嵌入矩阵的维度。

`context_length`: int 最大上下文长度，用于确定RoPE正弦和余弦缓冲区的维度。

`num_layers`: int 所使用的Transformer块数量。

为验证你的实现是否符合我们提供的测试用例，你首先需要实现测试
adapter位于[adapters.run_transformer_lm]。然后运行`uv run pytest -k test_transformer_lm`以
测试你的实现。

交付要求：一个能通过上述测试的Transformer语言模型模块。

资源核算

理解Transformer各部分如何消耗计算资源和内存十分有用。我们将逐步进行基础FLOPs核算。
Transformer中绝大多数FLOPs来自矩阵乘法，因此我们的核心方法十分简单：

1. 写出Transformer前向传播中所有的矩阵乘法。
2. 将每项矩阵乘法转换为所需FLOPs。第二步中，以下事实将很有用：

规则：给定 $\in \mathbb{R}^{n \times m}$ 和 $\in \mathbb{R}^{m \times k}$ ，矩阵乘积需 $2nmk$ 个FLOPs。

要理解这一点，请注意 $(AB)_{ij} = \sum_k A_{ik} B_{kj}$ ，而该点积需 k 次加法和 k 次乘法（共 $2k$ 个FLOPs）。由于矩阵乘积有 $n \times k$ 个元素，因此总FLOPs数为 $(2k)(nk) = 2n^2k$ 。

现在，在解答下一题之前，建议逐一梳理Transformer块及Transformer语言模型的各个组件，并列出所有矩阵乘法及其对应的FLOPs开销。

题目 (transformer_accounting) : Transformer语言模型资源核算 (5分)

- (a) 考虑一个采用本作业架构的GPT-2 XL尺寸模型，其配置如下：

```
vocab_size: 50,257
context_length: 1,024
num_layers: 48
d_model: 1,600
num_heads: 25
d_ff: 4,288 (最接近8的64的倍数)  $\frac{4}{3} \times 1,600$ 
```

假设我们按此配置构建模型，该模型共有多少可训练参数？若每个参数以单精度浮点数表示，仅加载该模型需要多少内存？

交付要求：一段一至两句话的回答。

- (b) 列出完成我们GPT-2 XL结构模型一次前向传播所需的全部矩阵乘法。这些矩阵乘法总共需要多少FLOPs？假设输入序列包含`context_length`个token。

交付要求：列出所有矩阵乘法（含说明）及所需总FLOPs数量。

(c) 基于上述分析，模型中哪些部分消耗的FLOPs最多？交付要求：一段一至两句话的回答。

(d) 使用GPT-2 small (12层、d_model=768、12个注意力头)、GPT-2 medium (24层、d_model=1024、16个注意力头) 和GPT-2 large (36层、d_model=1280、20个注意力头) 重复上述分析。随着模型尺寸增大，Transformer语言模型中哪些部分在总FLOPs中所占比例上升或下降？

交付要求：对每个模型，分别列出各组件及其对应的FLOPs（占单次前向传播总FLOPs的比例）。此外，用一至两句话描述模型尺寸变化如何影响各组件FLOPs的占比。

(e) 采用GPT-2 XL并将其上下文长度增至16,384。一次前向传播所需的总FLOPs如何变化？各模型组件的FLOPs占比如何变化？

交付要求：一段一至两句话的回答。

4 训练一个Transformer语言模型

我们现在已具备数据预处理（通过分词器）和模型（Transformer）的步骤。接下来需要构建全部支持训练的代码，包括以下内容：

- 损失：我们需要定义损失函数（交叉熵）。
- 优化器：我们需要定义用于最小化该损失的优化器（AdamW）。
- 训练循环：我们需要完整的支持基础设施，包括数据加载、检查点保存及训练管理。

4.1 交叉熵损失

回顾可知，Transformer语言模型为每个长度为+1的序列定义了分布(+1 | 1:)，其中=1,...。给定一个由长度为+1的序列组成的训练集，我们定义标准交叉熵（负对数似然）损失函数：

$$\ell(\cdot) = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^m -\log(\pi_j | 1:). \quad (16)$$

（注意：Transformer的一次前向传播会同时输出所有=1,...的(+1 | 1:)。）

特别地，Transformer为每个位置计算logits $\in \text{vocab_size}$ ，从而得到：

$$\pi_j | 1: = \text{softmax}(\text{logits}_j)[+1] = \frac{\exp(\text{logits}_j[+1])}{\sum_{i=1}^{\text{vocab_size}} \exp(\text{logits}_j[i])}. \quad (17)$$

交叉熵损失通常针对logits向量 $\in \text{vocab_size}$ 和目标定义。

+1.7

6 注意， i 表示向量中索引为的元素值。

7 这对应于+1处的狄拉克 δ 分布与预测的softmax()分布之间的交叉熵。

实现交叉熵损失需谨慎处理数值问题，这与 softmax 的实现类似。

问题 (cross_entropy) : 实现交叉熵 (1 分)

交付要求: 编写一个计算交叉熵损失的函数, 该函数接收预测 logits () 和目标 (+1), 并计算交叉熵 $\ell = -\log\text{softmax}()[+1]$ 。您的函数应满足以下要求:

- 减去最大元素以保证数值稳定性。
- 在可能的情况下, 抵消 log 和 exp 运算。
- 处理任意额外的批次维度, 并返回批次上的平均值。与第 3.2 节一样, 我们假设类批次维度始终位于词汇表维度之前。

实现 [adapters.run_cross_entropy], 然后运行 `uv run pytest -k test_cross_entropy` 来测试您的实现。

困惑度

交叉熵足以用于训练, 但在评估模型时, 我们也希望报告困惑度。对于长度为 m 的序列, 若其交叉熵损失为 $\ell_1, \dots, \ell_m$:

$$\text{困惑度} = \exp\left(-\frac{1}{m} \sum_{i=1}^m \ell_i\right). \quad (18)$$

4.2 SGD 优化器

在已定义损失函数的基础上, 我们将开始探索优化器。最简单的基于梯度的优化器是随机梯度下降 (SGD)。我们从随机初始化的参数 θ_0 开始, 然后对每一步 $t = 0, \dots, T-1$ 执行如下更新:

$$\theta_{t+1} \leftarrow \theta_t - \eta \nabla_{\theta} \ell(\theta_t; \mathbf{x}_t, \mathbf{y}_t), \quad (19)$$

其中 $\mathbf{x}_t, \mathbf{y}_t$ 是从数据集 $\mathcal{D}$ 中随机采样的一个批次数据, 学习率 η 和批次大小 $|\mathcal{B}|$ 均为超参数。

4.2.1 在 PyTorch 中实现 SGD

为实现我们的优化器, 我们将继承 PyTorch 的 `torch.optim.Optimizer` 类。Optimizer 子类必须实现两个方法:

`def __init__(self, params, ...)` 应初始化您的优化器。此处 `params` 是待优化参数的集合 (或参数组, 当用户希望对模型不同部分使用不同超参数 (例如学习率) 时)。请确保将 `params` 传给基类的 `__init__` 方法, 以便在 `step` 中使用这些参数。您可以根据优化器需要接受额外参数 (例如学习率就是常见选项), 并将它们作为字典传给基类构造函数, 其中键为您自定义的参数名称 (字符串)。

`def step(self)` 应对参数执行一次更新。在训练循环中, 该方法将在反向传播后被调用, 因此可访问上一批次的梯度。该方法应遍历每个参数张量 `p` 并就地修改它们, 即设置 `p.data` (它保存着对应参数的张量)。

与该参数关联的张量，基于其梯度 `p.grad`（若存在），即损失对该参数的梯度所对应的张量。

PyTorch 优化器 API 存在若干细节，因此通过示例解释更为清晰。为使示例更丰富，我们将实现 SGD 的一种微小变体：其学习率随训练进程逐渐衰减，起始学习率为 η ，并在后续训练中逐步采取更小的步长：

$$\eta_{t+1} = \frac{\eta_t}{\sqrt{t+1}} \nabla L(\theta_t) \quad (20)$$

让我们看看这种 SGD 版本如何作为 PyTorch Optimizer 实现：

```
from collections.abc import Callable, Iterable
from typing import Optional
import torch
import math

class SGD(torch.optim.Optimizer):
    def __init__(self, params, lr=1e-3):
        if lr < 0:
            raise ValueError(f"Invalid learning rate: {lr}")
        defaults = {lr: lr}

        super().__init__(params, defaults)

    def step(self, closure: Optional[Callable] = None):
        loss = None if closure is None else closure()
        for group in self.param_groups:
            lr = group[lr] # 获取学习率。
            for p in group[params]:
                if p.grad is None:
                    continue

                state = self.state[p] # 获取与 p 关联的状态。
                t = state.get('t', 0) # 从状态中获取迭代次数，若不存在则默认为 0。
                grad = p.grad.data # 获取损失关于 p 的梯度。
                p.data -= lr / math.sqrt(t + 1) * grad # 就地更新权重张量。
                state['t'] = t + 1 # 递增迭代次数。

        return loss
```

在 `__init__` 中，我们将参数以及默认超参数传递给基类构造函数（参数可能以分组形式传入，每组具有不同的超参数）。若参数仅为单一 `torch.nn.Parameter` 对象集合，则基类构造函数将创建单个参数组，并为其分配默认超参数。随后，在 `step` 方法中，我们遍历每个参数组，再遍历该组中的每个参数，并应用公式 20。此处，我们将迭代次数作为与每个参数关联的状态保存：首先读取该值，将其用于梯度更新，然后递增该值。API 规定用户可在优化器步骤前传入一个可调用闭包（callable closure）以重新计算损失。我们在所实现的优化器中无需此功能，但为符合 API 要求仍予以保留。要观察其实际运行效果，可参考以下最小训练循环示例：

```
weights = torch.nn.Parameter(5 * torch.randn((10, 10)))
opt = SGD([weights], lr=1)

for t in range(100):
    opt.zero_grad() # 重置所有可学习参数的梯度。
    loss = (weights**2).mean() # 计算标量损失值。
    print(loss.cpu().item())
```

```
loss.backward() # 执行反向传播，计算梯度。□ opt.step() # 执行优化器步骤。
```

这是典型的训练循环结构：在每次迭代中，我们将计算损失并执行一次优化器步骤。在训练语言模型时，我们的可学习参数来自模型（在 PyTorch 中，`m.parameters()` 提供了这一参数集合）。损失将在一个采样得到的数据批次上计算，但训练循环的基本结构保持不变。

问题 (learning_rate_tuning)：调整学习率（1 分）

正如我们将看到的，影响训练效果最显著的超参数之一是学习率。让我们在我们的玩具示例中实际观察这一点。使用另外三个学习率值（`1e1`、`1e2` 和 `1e3`）运行上面的 SGD 示例，仅进行 10 次训练迭代。每个学习率下的损失会发生什么变化？它是衰减得更快、更慢，还是发散（即在训练过程中持续增大）？

交付内容：用一至两句话描述你观察到的行为。

4.3 AdamW

现代语言模型通常采用比随机梯度下降（SGD）更复杂的优化器进行训练。近期广泛使用的多数优化器均为 Adam 优化器 [D. P. Kingma 等人，2015] 的变体。我们将使用 AdamW [I. Loshchilov 等人，2019]，其在近期工作中被广泛应用。AdamW 对 Adam 进行了改进，通过在每次迭代中将参数向零拉拢的方式引入权重衰减（即解耦权重衰减与梯度更新），从而改善正则化效果。我们将按 I. Loshchilov 等人 [23] 文中算法 2 所述实现 AdamW。

AdamW 是有状态的：对每个参数，它持续跟踪其一阶与二阶矩的滑动估计值。因此，AdamW 以额外内存开销为代价，提升了训练稳定性与收敛性。除学习率外，AdamW 还包含一对控制矩估计更新的超参数 $(1, 2)$ 以及一个权重衰减率。典型应用中 $(1, 2)$ 设为 $(0.9, 0.999)$ ，但 LLaMA [H. Touvron 等人，2023] 和 GPT-3 [T. B. Brown 等人，2020] 等大语言模型常采用 $(0.9, 0.95)$ 。该算法可表述如下，其中 ϵ 为极小值（例如 10^{-8} ），用于在出现极小值时提升数值稳定性：

算法 1: AdamW 优化器

```
1 init() 初始化可学习参数
    2  $\leftarrow 0$  一阶矩向量初始值; 形状与相同
    3  $\leftarrow 0$  二阶矩向量初始值; 形状与相同
4 for = 1, ..., do
5 采样一批数据
6  $\leftarrow \nabla \ell(\cdot)$  计算损失梯度
7  $\leftarrow \sqrt{1 - 2^{-1}}$  计算第次迭代的调整后
8  $\leftarrow$  应用权重衰减
9  $\leftarrow 1 + (1 - 1)$  更新一阶矩估计
10  $\leftarrow 2 + (1 - 2)^2$  更新二阶矩估计
11  $\leftarrow$   $\frac{\sqrt{\cdot}}{\sqrt{\cdot}}$  应用经动量调整的权重更新
12 结束循环
```

算法 1: AdamW 优化器伪代码。

注意从 1 开始计数。你现在将实现该优化器。

题目 (adamw) : 实现 AdamW (2 分)

交付内容: 将 AdamW 优化器实现为 `torch.optim.Optimizer` 的子类。你的类应在 `__init__` 中接收学习率, 以及、和超参数。为便于维护状态, 基类 `Optimizer` 提供了字典 `self.state`, 该字典将 `nn.Parameter` 对象映射到一个字典中, 用于存储该参数所需的任何信息 (对 AdamW 而言即动量估计值)。实现 `[adapters.get_adamw_cls]` 并确保

确保通过 `uv run pytest -k test_adamw` 测试。

题目 (adamw_accounting) : 使用 AdamW 训练的资源核算 (2 分)

我们来计算运行 AdamW 所需的内存与计算量。假设所有张量均使用 `float32` 格式。

(a) 运行 AdamW 所需峰值内存为多少? 请根据参数、激活值、梯度及优化器状态的内存占用进行分解。答案请用 `batch_size` 及模型超参数 (`vocab_size`、

`context_length`、`num_layers`、`d_model`、`num_heads`)。假设 `ff = 8 \times \text{model}`。

为简化起见, 在计算激活值内存占用时, 仅考虑以下组件:

- Transformer 块

RMSNorm(s)

多头自注意力子层: 投影、 $\top$ 矩阵乘法、softmax、加权值求和、输出投影。

逐位置前馈网络 (SwiGLU) : 1、2、门分支上的 SiLU 激活函数、逐元素乘积、3

- 最终 RMSNorm 层
- 输出嵌入层
- logits 上的交叉熵

交付内容：参数、激活值、梯度及优化器状态各自的代数表达式，以及总内存占用表达式。

(b) 将你的答案代入 GPT-2 XL 规模模型，得到仅依赖 `batch_size` 的表达式。在不超过 80GB 显存的前提下，你能使用的最大批量大小是多少？

交付内容：形如 `·batch_size +` 的表达式，其中、为数值，另附最大批量大小数值。

(c) 执行一次 AdamW 更新需多少浮点运算量 (FLOPs)？交付

内容：一个代数表达式，并附简要说明。

(d) 模型浮点利用率 (MFU) 定义为观测吞吐量 (每秒 token 数) 与硬件理论峰值浮点运算吞吐量之比。

[A. Chowdhery 等人, 2022]。NVIDIA H100 GPU 在 float32 (实际为 TensorFloat-32, 现实中为 bfloat16) 运算下的理论峰值算力为 495 万亿次浮点运算每秒 (teraFLOP/s)。假设你可达到 50% 的模型浮点利用率 (MFU)，那么在单块 H100 上以批量大小 1024 训练 GPT-2 XL 共 400K 步需要多长时间？参照 J. Kaplan 等人 [25] 及 J. Hoffmann 等人 [26] 假设反向传播的浮点运算量 (FLOPs) 为前向传播的两倍。

交付内容：训练所需小时数，并附简要说明。

4.4 学习率调度

导致损失最快下降的学习率值通常在训练过程中发生变化。在 Transformer 模型训练中，通常采用学习率调度策略：初始使用较大的学习率，在训练初期实现更快的更新，并随模型训练逐步衰减至较小值。⁸ 本作业中，我们将实现用于训练 LLaMA 的余弦退火调度策略。

[H. Touvron 等人, 2023]。

调度器本质上是一个函数，它接收当前步数及其他相关参数（例如初始与最终学习率），并返回在第步用于梯度更新的学习率。最简单的调度策略是常数函数，即对任意均返回相同的学习率。

余弦退火学习率调度器接收以下参数：(i) 当前迭代步数，(ii) 最大学习率 max ，(iii) 最小（最终）学习率 min ，(iv) 预热迭代步数，以及 (v) 余弦退火的最终迭代步数。在第次迭代时的学习率为：

(热身) 若 $w < w_{\text{warm}}$ ，则 $\eta = \eta_{\text{min}}$ 。

(余弦退火) 若 $w_{\text{warm}} \leq w \leq w_{\text{cos}}$ ，则 $\eta = \eta_{\text{min}} + \frac{1}{2}(1 + \cos(\frac{w - w_{\text{warm}}}{w_{\text{cos}} - w_{\text{warm}}}))(\eta_{\text{max}} - \eta_{\text{min}})$ 。

(退火后) 若 $w > w_{\text{cos}}$ ，则 $\eta = \eta_{\text{min}}$ 。

问题（学习率调度）：实现带预热的余弦学习率调度（1 分）

⁸ 有时会采用一种学习率回升（重启）的调度策略，以帮助跳出局部极小值。

编写一个函数，接收 `max`、`min`、`lr`，并依据上述调度器定义返回学习率。随后实现 `[adapters.get_lr_cosine_schedule]` 并确保它通过 `uv run pytest -k test_get_lr_cosine_schedule` 测试。

4.5 梯度裁剪

训练过程中，有时会遇到产生极大梯度的训练样本，从而导致训练不稳定。为缓解此问题，实践中常用的一种技术是梯度裁剪。其思想是在每次反向传播后、执行优化器步进前，对梯度范数施加限制。给定全部参数的梯度，我们计算其 ℓ_2 范数 $\|\mathbf{g}\|_2$ 。若该范数小于最大值，则保持不变；否则，将 $\mathbf{g}$ 按比例缩放为

$\frac{\mathbf{g}}{\|\mathbf{g}\|_2} \cdot \text{max_norm}$ （其中一小部分 max_norm ，如 10^{-6} ，用于保证数值稳定性）。注意，最终范数将略小于 max_norm 。

问题（梯度裁剪）：实现梯度裁剪（1 分）

编写一个实现梯度裁剪的函数。该函数应接收一组参数及最大 ℓ_2 范数。它需就地修改每个参数的梯度。使用 `max_norm = 10^{-6}`（PyTorch 默认值）。随后，实现适配器 `[adapters.run_gradient_clipping]` 并确保它通过 `uv run pytest -k test_gradient_clipping` 测试。

5 训练循环

我们现在终于将此前构建的主要组件整合起来：分词后的数据、模型以及优化器。

5.1 数据加载器

经分词处理后的数据（例如您在 `tokenizer_experiments` 中准备的数据）是一串连续的 token 序列 $(1, \dots)$ 。尽管原始数据可能由独立文档组成（例如不同网页或源代码文件），一种常见做法是将所有文档拼接为单一 token 序列，并在各文档之间添加分隔符（例如 `<|endoftext|>` token）。

数据加载器将其转换为一批批数据流，每个 batch 包含 n 个长度为 m 的序列，并配以对应长度为 m 的下一 token 目标。例如，当 $n = 1$ 、 $m = 3$ 时， $[(2,3,4), [3,4,5]]$ 即为一个可能的 batch。

以这种方式加载数据，出于多个原因简化了训练过程。首先，对任意满足 $1 \leq m \leq \text{vocab_size}$ 的 m ，均可构成有效的训练序列，因此采样训练序列极为简单。由于所有训练序列长度相同，无需对输入序列进行填充，从而提升了硬件利用率（亦可通过增大 batch size 实现）。最后，我们也不必一次性将完整数据集载入内存即可采样训练数据，这使得处理原本无法装入内存的大规模数据集变得轻而易举。

问题（数据加载）：实现数据加载（2 分）

交付成果：编写一个函数，接收一个 NumPy 数组（含整型 token ID）、`batch_size`、`context_length` 及 PyTorch 设备字符串（例如 `'cpu'` 或 `'cuda:0'`），并返回一对张量：采样的输入序列及其对应的下一 token 目标。两个

张量形状应为 (batch_size, context_length)，包含 token ID，并且两者均需置于指定设备上。为通过我们提供的测试验证您的实现，您首先需实现 [adapters.run_get_batch] 测试适配器，然后运行 `uv run pytest -k test_get_batch` 进行测试。

低资源提示：CPU 或 Apple Silicon 上的数据加载

若您计划在 CPU 或 Apple Silicon 上训练语言模型，则需将数据移至正确的设备上（同样，后续模型也应使用同一设备）。

若您使用 CPU，可采用 'cpu' 设备字符串；若您使用 Apple Silicon（M* 芯片），则可采用 'mps' 设备字符串。

有关 MPS 的更多内容，请参阅以下资源：

- <https://docs.pytorch.org/docs/stable/mps.html>
- <https://docs.pytorch.org/docs/stable/notes/mps.html>
- <https://developer.apple.com/documentation/metalperformanceshaders>

若数据集过大而无法全部载入内存怎么办？我们可以使用 Unix 系统调用 `mmap`，它将磁盘上的文件映射到虚拟内存，并在访问对应内存位置时按需加载文件内容。因此，您可假定整个数据集已驻留在内存中。NumPy 通过 `np.memmap`（或若原始数组由 `np.save` 保存，则在 `np.load` 中设置标志 `mmap_mode='r'`）实现了这一机制，它将返回一个类似 NumPy 数组的对象，当您访问其中元素时才按需加载。在训练期间对数据集（即 NumPy 数组）进行采样时，请务必以内存映射模式加载数据集（通过 `np.memmap` 或根据数组保存方式选择 `np.load` 的 `mmap_mode='r'` 标志）。同时请确保指定与待加载数组匹配的 `dtype`。显式验证内存映射数据是否正确（例如确认其值未超出预期词表大小）可能很有帮助。

5.2 检查点

除数据加载外，我们在训练过程中还需保存模型。运行任务时，我们常需恢复中途停止的训练（例如因作业超时、机器故障等原因）。即使一切顺利，我们也可能希望后续访问中间模型（例如事后分析训练动态、从不同训练阶段的模型中采样等）。

检查点应包含恢复训练所需的所有状态。我们至少需要能够还原模型权重。若使用有状态的优化器（如 AdamW），还需保存优化器的状态（例如 AdamW 中的动量估计）。最后，为恢复学习率调度，我们需要知道训练中断时的迭代次数。PyTorch 为此提供了便捷支持：每个 `nn.Module` 均提供 `state_dict()` 方法，返回一个包含所有可学习权重的字典；我们可通过其配套方法 `load_state_dict()` 后续恢复这些权重。`torch.optim.Optimizer` 同样如此。最后，`torch.save(obj, dest)` 可将一个对象（例如包含张量及其他常规 Python 对象（如整数）的字典）保存至文件（路径）或类文件对象，之后可通过 `torch.load(src)` 将其重新载入内存。

问题（检查点）：实现模型检查点功能（1 分）

实现以下两个函数以加载和保存检查点：

def save_checkpoint(model, optimizer, iteration, out) 应将以下全部状态转储至
将 model、optimizer 和 iteration 的全部状态写入类文件对象 out 中。您可以调用 model 和 optimizer 的 state_dict 方法获取其对应状态，并使用 torch.save(obj, out) 将 obj 写入 out（PyTorch 在此处支持路径或类文件对象）。通常做法是令 obj 为一个字典，但只要后续能成功加载检查点，您也可采用任意格式。

该函数期望接收以下参数：

model: torch.nn.Module
optimizer: torch.optim.Optimizer
iteration: int
out: str | os.PathLike | typing.BinaryIO | typing.IO[bytes]

def load_checkpoint(src, model, optimizer) 应从 src（路径或类文件对象）加载检查点，然后从中恢复模型与优化器的状态。您的函数应返回保存在检查点中的迭代次数。您可以使用 torch.load(src) 来恢复您在 save_checkpoint 实现中所保存的内容，并调用模型与优化器各自的 load_state_dict 方法，将其恢复至先前状态。

该函数期望接收以下参数：

src: str | os.PathLike | typing.BinaryIO | typing.IO[bytes]
model: torch.nn.Module
optimizer: torch.optim.Optimizer

实现 [adapters.run_save_checkpoint] 和 [adapters.run_load_checkpoint] 适配器，并确保它们通过 uv run pytest -k test_checkpointing 测试。

5.3 训练循环

现在，终于到了将你实现的所有组件整合进主训练脚本的时候。为便于使用不同超参数启动训练任务（例如通过命令行参数传入），值得投入精力优化脚本结构——因为你后续将多次执行此类操作，以研究不同选择对训练的影响。

问题（training_together）：整合全部组件（4分）

交付内容：编写一个脚本，运行训练循环以在用户提供的输入上训练你的模型。具体而言，我们建议你的训练脚本至少支持以下功能：

- 支持配置和控制各类模型及优化器超参数。
- 使用 np.memmap 实现大型训练与验证数据集的内存高效加载。
- 将检查点序列化至用户指定路径。

- 定期记录训练与验证性能（例如输出至控制台和/或外部服务如Weights and Biases）⁹

6 生成文本

在能够训练模型之后，我们所需的最后一块拼图便是从模型生成文本的能力。回顾一下，语言模型接收一个（可能批处理的）长度为sequence_length的整数序列，并输出一个尺寸为(sequence_length, vocab_size)的矩阵，其中序列中每个位置对应一个概率分布，用于预测该位置之后的下一个词元。接下来，我们将编写若干函数，将此输出转化为一种新序列的采样方案。

Softmax

按标准惯例，语言模型输出为最终线性层（即logits）的输出，因此我们需要通过softmax运算将其转换为归一化的概率分布——该运算已在公式10中介绍过。

解码

为从模型生成文本（即解码），我们将向模型提供一段前缀词元序列（即提示），并要求其输出一个覆盖整个词表的概率分布，以预测序列中的下一个词元。随后，我们从该词表分布中采样，以确定下一个输出词元。

具体而言，解码过程的每一步应接收一个序列1...，并依据下述公式返回一个词元+1：

$$p(y_{t+1} = i | 1...) = \frac{\exp(\text{logits}_{t,i})}{\sum_j \exp(\text{logits}_{t,j})} \quad (21)$$

$$= \text{TransformerLM}(1...)_{t,i} \in \mathbb{R}^{\text{vocab_size}} \quad (22)$$

其中TransformerLM是我们的模型，其输入为长度为sequence_length的序列，并输出一个尺寸为(sequence_length, vocab_size)的矩阵；我们取该矩阵的最后一个元素，因为我们关注的是第 sequence_length 个位置处的下一词元预测。

由此得到一个基础解码器：反复从这些单步条件分布中采样（将此前已生成的输出词元追加至下一步解码时间步的输入中），直至生成结束序列标记<|endoftext|>（或达到用户指定的最大生成词元数）。

解码技巧

我们将对小型模型展开实验，而小型模型有时会生成质量极低的文本。两种简单的解码技巧有助于缓解这些问题。第一种是温度缩放：我们通过温度参数 修改softmax，新softmax定义如下：

$$\text{softmax}(x, \tau) = \frac{\exp(x/\tau)}{\sum_{j=1}^{\text{vocab_size}} \exp(x_j/\tau)} \quad (23)$$

注意，当 $\tau \rightarrow 0$ 时， x 中最大元素将占据主导地位，此时softmax输出趋近于集中在该最大元素上的独热向量。

⁹wandb.ai

其次，另一种技巧是核采样（或称Top-p采样），即通过截断低概率词元来修改采样分布。设 π 是由（经温度缩放后的）softmax输出的概率分布，其大小为vocab_size。以超参数 p 进行核采样时，下一词元按如下公式生成：

$$(\pi + 1 = |) = \begin{cases} \frac{q_j}{\sum_{i \in ()} q_i} & \text{若 } j \in () \\ 0 & \text{否则} \end{cases} \quad (24)$$

其中 $()$ 是满足 $\sum$ 的最小索引集合。

$\sum_{i \in ()} q_i \geq p$ 。该量可轻松计算。首先按概率大小对分布进行排序，然后选取概率最大的词表元素，直至累积概率达到目标值。

问题（解码）：解码（3分）

交付内容：实现一个从你的语言模型解码的函数。我们建议你支持以下功能：

- 为用户提供提示（即输入一段文本1...），并采样生成补全文本，直至遇到<|endoftext|>标记。
- 允许用户控制生成词元的最大数量。
- 给定期望的温度值，在采样前对预测的下一词分布应用softmax温度缩放。
- Top-p采样（[A. Holtzman 等, 2020]，亦称核采样），给定用户指定的阈值。

7 实验

现在是时候整合所有内容，在预训练数据集上训练（小型）语言模型了。

7.1 实验运行方式与交付内容

理解Transformer架构组件设计原理的最佳方式，是亲自修改并运行它。动手实践无可替代。

为此，快速、一致地开展实验，并记录所做工作，至关重要。为实现快速实验，我们将基于一个小规模模型（约1700万总参数）和一个简单数据集（TinyStories）运行大量实验。为确保一致性，你将以系统化方式对组件进行消融分析并调整超参数；为保留记录，我们将要求你提交实验日志以及每个实验对应的学习曲线。

为便于提交损失曲线，请确保定期评估验证损失，并同时记录步数与真实时间。你可能会发现Weights and Biases等日志基础设施很有帮助。

题目（experiment_log）：实验日志记录（3分）

为你的训练与评估代码构建实验追踪基础设施，以便追踪实验过程及相对于梯度步数和真实时间的损失曲线。

交付成果：为本节以下作业题所设计的实验日志基础设施代码，以及一份实验日志（即你所尝试的所有内容的文档记录）。

7.2 TinyStories

我们将从一个非常简单的数据集（TinyStories；R. Eldan等人[1]）开始，模型在此数据集上训练很快，且能展现出一些有趣的行为。获取该数据集的说明见第1节。该数据集的一个示例如下所示。

示例（tinystories_example）：TinyStories中的一个例子

从前有个叫本的小男孩。本喜欢探索身边的世界。他看到了许多奇妙的事物，比如商店里陈列的漂亮花瓶。有一天，本在店里散步时，偶然发现了一个非常特别的花瓶。本一看到它就惊呆了！他喊道：“哇，这真是一个令人惊叹的花瓶！我能买下它吗？”店主微笑着回答：“当然可以。你可以把它带回家，向所有朋友展示它的神奇之处！”于是本把花瓶带回了家，为此感到无比自豪！他邀请朋友们过来，向他们展示了这个神奇的花瓶。所有朋友都觉得这个花瓶很美，难以相信本竟如此幸运。就这样，本在店里找到了一个神奇的花瓶！

7.2.1 超参数调优

我们将为你提供一些基础超参数作为起点，并请你为其余超参数寻找表现良好的设置。

词表大小：10000。典型的词表大小通常在几万到几十万之间。你应该尝试不同取值，观察词表及模型行为的变化。

上下文长度：256。TinyStories这类简单数据集可能无需很长的序列长度，但后续处理OpenWebText数据时，你可能需要调整该值。可尝试改变该值，观察其对每次迭代运行时间和最终困惑度的影响。

d_model：512。这一数值略小于许多小型Transformer论文中常用的768维，但能加快运算速度。

d_ff：1344。这约为d_model的8倍。³模型维度d_ff：1344。这大约是d_model的8倍。

RoPE theta参数 Θ ：10000。

层数与注意力头数：4层、16个头。这样总共约有1700万非嵌入参数，属于一个规模较小的Transformer模型。

处理的总token数为327,680,000（你的批量大小×总步数×上下文长度之积应大致等于该数值）。

你应该通过反复试验，为以下其他超参数寻找合适的默认值：

学习率、学习率预热、其他AdamW超参数（ β 、 β ）以及权重衰减。你需要自行确定这些超参数的合适默认值。可在D. P. Kingma等人[22]的论文中找到此类超参数的典型取值。

7.2.2 整合各部分

现在你可以将所有部分整合起来：获取已训练的BPE分词器，对训练数据集进行分词，并将其代入你编写好的训练循环中。重要提示：若你的实现正确且高效，上述超参数应在1块B200 GPU上带来约20–30分钟的运行时间。

GPU。若你的运行时间明显更长，请检查数据加载、检查点保存或验证损失计算代码是否成为瓶颈，并确认你的实现已正确批处理。

7.2.3 调试模型架构的技巧与窍门

我们强烈建议熟悉集成开发环境（IDE）内置的调试器（例如VSCode/Zed），这将比仅靠print语句调试节省大量时间。如果你使用纯文本编辑器，可考虑使用ipdb等工具。调试模型架构时，其他一些良好实践包括：

- 开发任何神经网络架构时，一个常见初步步骤是让模型过拟合单个小批量数据。若你的实现正确，应能快速将训练损失降至接近零。
- 在模型各组件中设置调试断点，检查中间张量的形状，确保其符合预期。
- 监控激活值、模型权重和梯度的范数，确保它们既不爆炸也不消失。

题目 (learning_rate)：调整学习率 (2 B200小时) (3分)

学习率是最关键的待调超参数之一。请基于你已训练的基础模型，回答以下问题：

(a) 对学习率进行超参数扫描，并报告最终损失（若优化器发散，则注明发散）。

交付成果：对应多个学习率的学习曲线图，并说明你的超参数搜索策略。

交付成果：在TinyStories数据集上验证损失（按token计算）不超过1.45的模型。

低资源提示：先在CPU或Apple Silicon设备上训练若干步。

如果你在CPU或MPS设备上运行，请将处理的总token数减少至40,000,000，这已足以生成合理流畅的文本。你也可以将目标验证损失从1.45上调至2.00。

我们在配备M4 Max芯片和36 GB内存的设备上，使用调优后的学习率运行解决方案代码，采用批量大小 $\times$ 总步数 $\times$ 上下文长度 $= 32 \times 5000 \times 256 = 40,960,000$ 个token的配置，在CPU上耗时1小时22分钟，在MPS上耗时36分钟。在第5000步时，验证损失为1.80。

一些额外提示：

- 在使用训练步数时，建议调整余弦学习率衰减调度，使其衰减过程（即达到最小学习率）恰好在第步终止。
- 使用MPS时，请勿启用TF32内核，即不要设置

```
torch.set_float32_matmul_precision('high')
```

就像你在CUDA设备上所做的那样。我们曾尝试在MPS上启用TF32内核（PyTorch 2.9.0），但发现该后端有时会静默调用存在缺陷的内核，导致训练不稳定。

- 你可以使用`torch.compile`对模型进行即时编译（JIT），从而加快训练速度。具体如下：

在CPU上，请使用以下方式编译你的模型：

```
model = torch.compile(model)
```

在MPS设备上，你可以通过以下方式在一定程度上优化反向传播过程：

```
model = torch.compile(model, backend=aot_eager)
```

截至PyTorch 2.9.0版本，Inductor编译在MPS后端上尚不支持。

(b) 民间经验认为，最佳学习率处于稳定性的临界点。请探究学习率开始发散的临界点与你找到的最佳学习率之间的关系。

交付成果：学习率递增的学习曲线图，其中至少包含一次发散的运行，并分析该现象与收敛速度之间的关系。

现在让我们改变批量大小，观察训练过程会发生什么变化。批量大小非常重要——它能让我们通过执行更大的矩阵乘法来提高GPU的利用效率，但是否批量大小越大越好呢？让我们运行一些实验来找出答案。

问题（batch_size_experiment）：批量大小变化实验（1 B200 小时）（1 分）

将你的批量大小从 1 逐步调至 GPU 内存上限。中间至少尝试几个批量大小，包括典型的 64 和 128。

交付成果：不同批量大小运行的学习曲线。必要时应重新优化学习率。

交付成果：用几句话讨论你关于批量大小及其对训练影响的发现。

手握解码器后，我们现在可以生成文本了！我们将从模型中生成文本并评估其质量。作为参考，你得到的输出至少应与下方示例一样好。

示例（ts_generate_example）：TinyStories 语言模型的样本输出

从前，有一个名叫莉莉的漂亮女孩。她特别爱吃口香糖，尤其是那种大大的黑色口香糖。一天，莉莉的妈妈让她帮忙做晚饭。莉莉兴奋极了！她很喜欢帮妈妈。莉莉的妈妈煮了一大锅汤当晚餐。莉莉非常开心，说：“谢谢妈妈！我爱你。”她帮妈妈把汤倒进一个大碗里。晚饭后，莉莉的妈妈又做了些美味的汤。莉莉非常喜欢！她说：“谢谢妈妈！这汤太好喝了！”妈妈微笑着说：“我很高兴你喜欢，莉莉。”她们做完饭后继续一起烹饪。故事结束。

低资源提示：在 CPU 或 Apple Silicon 设备上生成文本

如果你改用处理 4000 万 token 的低资源配置，则生成文本仍会类似英语，但不如上述结果流利。例如，我们在 4000 万 token 上训练的 TinyStories 语言模型的样本输出如下：

从前，有一个名叫苏的小女孩。苏有一颗非常喜爱的牙齿。那是她最好的头。一天，苏出去散步，遇见了一只瓢虫！她们成了好朋友，一起在小路上玩耍。

嘿，波莉！我们出去吧！蒂姆说道。苏抬头望向天空，发现很难找到一条闪亮起舞的路。她微笑着答应帮忙说话！

苏望着天空移动，那是什么。她

以下是具体的问题陈述及要求内容：

问题 (generate)：生成文本 (1 分)

使用你的解码器及已训练好的检查点，报告模型生成的文本。你可能需要调整解码器参数（如温度、top-p 等）以获得流利的输出。交付成果：至少 256 个 token 的文本输出（或直至首个 `<|endoftext|>` token），并简要评述该输出的流利程度，以及至少两个影响该输出质量优劣的因素。

7.3 消融实验与架构修改

理解 Transformer 的最佳方式是亲自修改它并观察其行为。接下来我们将进行若干简单消融实验与架构修改。

消融实验 1：层归一化

人们常说层归一化对 Transformer 训练的稳定性至关重要。但或许我们想冒险一试——移除每个 Transformer 块中的 RMSNorm，看看会发生什么。

问题 (layer_norm_ablation)：移除 RMSNorm 并训练 (0.5 B200 小时) (1 分)

从你的 Transformer 中移除所有 RMSNorm 并进行训练。在先前最优学习率下会发生什么？能否通过降低学习率获得稳定性？

交付成果：绘制一条移除 RMSNorm 后训练的学习曲线，以及一条对应最优学习率的学习曲线。

交付成果：用几句话评述 RMSNorm 的影响。

现在我们来考察另一种初看似乎随意的层归一化选择。前归一化 Transformer 块定义为

= + MultiHeadSelfAttention(RMSNorm()) (25)

= + FFN(RMSNorm())。 (26)

这是对原始 Transformer 架构为数不多的共识性修改之一，原始架构采用后归一化方法，

$$= \text{RMSNorm}(+ \text{MultiHeadSelfAttention}()) \quad (27)$$

$$= \text{RMSNorm}(+ \text{FFN}()). \quad (28)$$

让我们回归后归一化方法并观察结果。

问题 (pre_norm_ablation)：实现后归一化并训练 (0.5 B200 小时) (1 分)

将你的前归一化 Transformer 实现改为后归一化形式。使用后归一化模型进行训练并观察结果。

交付成果：绘制一条学习曲线，对比后归一化 Transformer 与前归一化 Transformer 的性能。

我们发现层归一化对 Transformer 的行为有重大影响，甚至其放置位置也至关重要。

消融实验 2：位置嵌入

接下来我们将考察位置嵌入对模型性能的影响。具体而言，我们将基础模型（含 RoPE）与完全不包含位置嵌入（NoPE）的情况进行对比。事实证明，仅解码器架构的 Transformer（即我们已实现的带因果掩码的架构）理论上无需显式提供位置嵌入，即可推断出相对或绝对位置信息 [Y.-H. H. Tsai 等，2019; A. Kazemnejad 等，2023]。我们现在将通过实证方式检验 NoPE 相对于 RoPE 的表现。



问题 (no_pos_emb)：实现 NoPE (0.5 B200 小时) (1 分)

修改你已实现 RoPE 的 Transformer，完全移除位置嵌入信息，并观察结果。

交付成果：绘制一条学习曲线，对比 RoPE 与 NoPE 的性能。

消融实验 3：SwiGLU 与 SiLU

接下来，我们将遵循 N. Shazeer [20] 的方法，通过对比 SwiGLU 前馈网络与仅使用 SiLU 激活但无门控线性单元（GLU）的前馈网络，来检验前馈网络中门控机制的重要性：

$$\text{FFNSiLU}() = 2 \text{ SiLU}(1). \quad (29)$$

回顾我们在 SwiGLU 实现中，将前馈层内部维度设为约 $\text{ff} = 8$ 。

³模型（同时确保 $\text{ff} \bmod 64 = 0$ ，以充分利用 GPU 张量核心）。在此消融基线中，你的 FFNSiLU 实现应将 ff 设为 $4 \times \text{model}$ ，以大致匹配默认 SwiGLU 前馈网络的参数量（后者使用三个而非两个权重矩阵）。

问题 (swiglu_ablation)：SwiGLU 与 SiLU 对比 (0.5 B200 小时) (1 分)

交付成果：绘制一条学习曲线，对比 SwiGLU 与 SiLU 前馈网络的性能，二者参数量大致相当。

交付成果：用几句话讨论你的发现。

低资源提示：GPU 资源有限的在线学生应在 TinyStories 数据集上测试各项修改。

在本作业剩余部分中，我们将转向规模更大、噪声更多的网页数据集（OpenWebText），尝试架构修改，并（可选）向课程排行榜提交结果。

在 OpenWebText 数据集上将语言模型训练至流利程度需要很长时间，因此我们建议 GPU 资源有限的在线学生继续在 TinyStories 数据集上测试各项修改（以验证损失作为评估性能的指标）。

7.4 在 OpenWebText 上运行

我们现在将转向一个更为标准的预训练数据集，该数据集由网络爬虫构建而成。OpenWebText [A. Gokaslan 等人, 2019] 的一小部分样本同样以单个文本文件形式提供：访问该文件的方法详见第 1 节。

以下是 OpenWebText 中的一个示例。请注意，这段文本更加真实、复杂且多样。你或许希望浏览训练数据集，以了解网络爬取语料库的训练数据形态。

示例 (owt_example)：OWT 中的一个示例

Baseball Prospectus 技术总监哈里·帕夫利迪斯在聘用乔纳森·贾奇时冒了一次险。

正如艾伦·施瓦茨在《数字游戏》中所写，美国文化中没有任何领域比棒球运动员的表现被更精确地计数、更热忱地量化。只需几次点击，你就能查到诺亚·辛德加德的快速球在飞向本垒板途中每分钟旋转超过 2100 圈；纳尔逊·克鲁兹在 2016 年所有合格击球员中拥有全联盟最高的平均击球初速；还有无数其他看似源自电子游戏或科幻小说的细节。数据洪流不断上涨，赋予了棒球文化中一个日益重要的角色：分析型业余爱好者。

这种赋权也伴随着额外审视——不仅针对测量指标本身，还针对指标背后的人员与出版物。帕夫利迪斯深知 Baseball Prospectus 在量化不完善时所引发的强烈反弹。他也清楚该网站的捕手评估指标亟待重构，而这项工作需要一位具备扎实学识的人——一位能够解决复杂统计建模问题的人——来完成。

他让我们毛骨悚然。哈里·帕夫利迪斯

帕夫利迪斯凭直觉认为，法官是根据后者的写作及其在场地赞助的棒球场活动中的互动而得出这一结论的。[...]

注意：本实验中，你可能需要重新调整学习率或批量大小等超参数。

题目（主实验）：在 OWT 上的实验（2 B200 小时）（2 分）

使用与 TinyStories 相同的模型架构和总训练步数，在 OpenWebText 上训练你的语言模型。该模型表现如何？

交付成果：你的语言模型在 OpenWebText 上的学习曲线。描述其损失值与 TinyStories 的差异——我们应如何解读这些损失值？

交付成果：OpenWebText 语言模型生成的文本，格式与 TinyStories 输出相同。该文本的流畅度如何？尽管模型结构与计算预算与 TinyStories 相同，为何输出质量反而更差？

7.5 自行修改 + 排行榜

恭喜你走到这一步！你已接近完成！接下来，你将尝试改进 Transformer 架构，并观察你的超参数与架构与其他同学相比的表现如何。

排行榜规则

除以下限制外，无其他约束：

运行时间：你的提交最多可在 B200 上运行 45 分钟。若使用 SLURM 或 Modal，你可能需要在提交脚本中强制执行该限制。

数据：你仅可使用我们提供的 OpenWebText 训练数据集。除此之外，你可以自由发挥。

若你希望获得一些实现思路，可参考以下资源：

- 最先进的开源大语言模型系列，例如 Llama 3 [A. Grattafiori 等人, 2024] 或 Qwen 2.5 [A. Yang 等人, 2024]。
- NanoGPT 速通仓库 (github.com/KellerJordan/modded-nanogpt)，社区成员在此发布了许多针对小规模语言模型预训练速通的有趣修改。例如，一种常见修改可追溯至原始 Transformer 论文，即联合输入和输出嵌入的权重（参见 A. Vaswani 等人 [8]（第 3.4 节）及 A. Chowdhery 等人 [16]（第 2 节））。若采用权重联合，你可能需要降低嵌入层/语言模型头初始化的标准差。

在尝试完整的 45 分钟运行前，建议先在 OpenWebText 的小规模子集或 TinyStories 上测试这些修改。

需要提醒的是，你在本次排行榜中发现效果良好的某些修改，未必能推广至更大规模的预训练任务。课程的“缩放定律”单元将进一步探讨这一问题。

题目（排行榜）：排行榜（10 B200 小时）（6 分）

你需依据上述排行榜规则训练一个模型，目标是在 0.75 B200 小时内将你的语言模型验证损失降至最低。

交付成果：最终记录的验证损失值、一条清晰展示横轴为实际运行时间（小于 45 分钟）的学习曲线，以及你所采取措施的说明。我们期望排行榜提交结果至少优于 5.0 损失值这一朴素基线。请在此处提交至排行榜：github.com/stanford-cs336/assignment1-basics-leaderboard。

参考文献

- [1] R. Eldan 和 Y. Li, 《TinyStories: 语言模型最小能小到什么程度仍能说出连贯的英语? 》, 2023 年。
- [2] A. Gokaslan、V. Cohen、E. Pavlick 和 S. Tellex, 《OpenWebText 语料库》, 2019 年。
- [3] R. Sennrich、B. Haddow 和 A. Birch, 《使用子词单元进行稀有词的神经机器翻译》, 载于《ACL 会议论文集》, 2016 年。
- [4] C. Wang、K. Cho 和 J. Gu, 《基于字节级子词的神经机器翻译》, 2019 年。
- [5] P. Gage, 《一种新的数据压缩算法》, 《C 用户期刊》, 第 12 卷, 第 2 期, 第 23–38 页, 1994 年 2 月。
- [6] A. Radford、J. Wu、R. Child、D. Luan、D. Amodei 和 I. Sutskever, 《语言模型是无监督多任务学习器》, 2019 年。
- [7] A. Radford、K. Narasimhan、T. Salimans 和 I. Sutskever, 《通过生成式预训练提升语言理解能力》, 2018 年。
- [8] A. Vaswani 等人, 《注意力机制就是全部》, 载于《NeurIPS 会议论文集》, 2017 年。
- [9] T. Q. Nguyen 和 J. Salazar, 《无泪 Transformer: 改进自注意力的归一化》, 载于《IWSLT 会议论文集》, 2019 年。
- [10] R. Xiong 等人, 《论 Transformer 架构中的层归一化》, 载于《ICML 会议论文集》, 2020 年。
- [11] J. L. Ba、J. R. Kiros 和 G. E. Hinton, 《层归一化》, 2016 年。
- [12] H. Touvron 等人, 《LLaMA: 开放且高效的基础语言模型》, 2023 年。
- [13] B. Zhang 和 R. Sennrich, 《均方根层归一化》, 载于《NeurIPS 会议论文集》, 2019 年。
- [14] A. Grattafiori 等人, 《Llama 3 模型群》。[在线资源]。可获取网址：<https://arxiv.org/abs/2407.21783>
- [15] A. Yang 等人, 《Qwen2.5 技术报告》, arXiv 预印本 arXiv:2412.15115, 2024 年。
- [16] A. Chowdhery 等人, 《PaLM: 利用 Pathways 扩展语言建模》, 2022 年。
- [17] D. Hendrycks 和 K. Gimpel, 《以高斯误差线性单元桥接非线性与随机正则化项》, 2016 年。
- [18] S. Elfwing、E. Uchibe 和 K. Doya, 《强化学习中用于神经网络函数逼近的 Sigmoid 加权线性单元》。[在线资源]。可获取网址：<https://arxiv.org/abs/1702.03118>
- [19] Y. N. Dauphin、A. Fan、M. Auli 和 D. Grangier, 《使用门控卷积网络进行语言建模》。[在线资源]。可获取网址：<https://arxiv.org/abs/1612.08083>
- [20] N. Shazeer, 《GLU 变体改进 Transformer》, 2020 年。

- [21] J. Su、Y. Lu、S. Pan、B. Wen 和 Y. Liu, 《RoFormer: 采用旋转位置嵌入增强的Transformer》, 2021 年。
- [22] D. P. Kingma 和 J. Ba, 《Adam: 一种随机优化方法》, 载于《ICLR 会议论文集》, 2015 年。
- [23] I. Loshchilov 和 F. Hutter, 《解耦权重衰减正则化》, 载于ICLR会议论文集, 2019年。
- [24] T. B. Brown 等, 《语言模型是少样本学习者》, 载于NeurIPS会议论文集, 2020年。
- [25] J. Kaplan 等, 《神经语言模型的缩放定律》, 2020年。
- [26] J. Hoffmann 等, 《训练计算最优的大语言模型》, 2022年。
- [27] A. Holtzman、J. Buys、L. Du、M. Forbes 和 Y. Choi, 《神经文本退化的奇异案例》, 载于ICLR会议论文集, 2020年。
- [28] Y.-H. H. Tsai、S. Bai、M. Yamada、L.-P. Morency 和 R. Salakhutdinov, 《Transformer解构: 透过核视角对Transformer注意力机制的统一理解》, 载于2019年自然语言处理实证方法会议暨第九届国际自然语言处理联合会议 (EMNLP-IJCNLP) 论文集, K. Inui、J. Jiang、V. Ng 和 X. Wan 编, 中国香港: 计算语言学协会, 2019年11月, 第4344–4353页。DOI: 10.18653/v1/D19-1443。
- [29] A. Kazemnejad、I. Padhi、K. Natesan、P. Das 和 S. Reddy, 《位置编码对Transformer模型长度泛化能力的影响》, 载于第三十七届神经信息处理系统大会, 2023年。[在线资源]。可获取网址:
<https://openreview.net/forum?id=Drrl2gcjzl>